Dynamic Timetable: A Role-Based Timetable Monitoring System with Automated Timetable Extraction and Natural-Language Queries

submitted in partial fulfillment of the requirement
for the Mini Project (Semester V) of

Third Year Bachelor of Technology
in
Computer Engineering

by

Shivam Mulani	2024300141
Gurshaan Nandrajog	2024300146
Harshvarddhan Waghela	2024300272

Group No. 86

under the guidance of

Prof. Anant Nimkar



Department of Computer Engineering
Bharatiya Vidya Bhavan's
Sardar Patel Institute of Technology
(Autonomous Institute Affiliated to University of Mumbai)
Munshi Nagar, Andheri-West, Mumbai-400058
University of Mumbai

Academic Year 2026–27

Approval Certificate

This is to certify that the Mini Project entitled "**Dynamic Timetable: A Role-Based Timetable Monitoring System with Automated Timetable Extraction and Natural-Language Queries**" by Shivam Mulani, Gurshaan Nandrajog and Harshvarddhan Waghela (Group No. 86) is approved for the partial fulfillment of the Mini Project of T.E. Computer Engineering (Semester V), Academic Year 2026–27, of the University of Mumbai.

Project Guide

(signature)

Name:

Date:

External Examiner

(signature)

Name:

Date:

Head of Department

Principal

Seal of the Institute

Declaration

We wish to state that the work embodied in this report titled "Dynamic Timetable: A Role-Based Timetable Monitoring System with Automated Timetable Extraction and Natural-Language Queries" forms our own contribution to the work carried out under the guidance of Prof. Anant Nimkar at the Sardar Patel Institute of Technology. We declare that this written submission represents our ideas in our own words and where others' ideas or words have been included, we have adequately cited and referenced the original sources. We also declare that we have adhered to all principles of academic honesty and integrity and have not misrepresented or fabricated or falsified any idea/data/fact/source in our submission.

Shivam Mulani	**Gurshaan Nandrajog**	**Harshvarddhan Waghela**
(signature)	(signature)	(signature)
UID: 2024300141	UID: 2024300146	UID: 2024300272

Date: Place: Mumbai

Abstract

In most colleges the class timetable is published as a multi-page PDF that every student and teacher has to read by hand. Finding the next class, a free room or a teacher's location means scanning dense grids full of abbreviations, lab sub-groups and footnote legends, and every revision of the timetable has to be re-checked manually. Clashes between teachers or rooms are usually discovered only when they happen.

This mini project builds **Dynamic Timetable**, a role-based timetable monitoring system made of a cross-platform mobile and web app (React Native with Expo) and a central back end (Python FastAPI and PostgreSQL). The Admin uploads the official timetable as PDF, Excel, Word or a photo. A background worker extracts every class cell with geometry-aware table parsing and OCR, expands the per-page legends, separates lab sub-groups, validates times and flags anything uncertain for review. Students, professors, the HOD and the Principal then see a dashboard (current class, next class, free periods) and can ask questions in plain English such as "Do I have classes after lunch on Tuesday?", "How many labs do I have this week?" or "Which day is my busiest?". A deterministic compositional planner turns each question into a typed query plan (an operation such as list, count, first/last, total hours or day-wise summary, combined with filters, a date or week, a time window and an ordering), which the server compiles into parameterised queries. It never generates SQL from text and cannot be talked into reading other users' data. Institutional and personal timetables are kept strictly separate, and the system automatically detects professor, room and batch clashes.

On the college's real 10-page timetable PDF the system extracted 453 class entries, of which 452 were verified automatically; the one remaining entry has no time printed in the PDF. It also reported exactly the two genuine clashes present in the document. The work is supported by 179 automated back-end tests, 20 mobile unit tests and end-to-end browser tests (24/24 and 21/21 passing). Measured single-user search latency was about 37–47 ms (median) on a 4-core development machine. The system has been deployed as a free online demo. Its limitations are that it has not been load tested on production hardware, push notifications need a store build, and extraction accuracy has not yet been measured against a hand-labelled ground truth.

Keywords: timetable extraction, table recognition, OCR, natural-language interface, query planning, role-based access control, conflict detection, React Native, FastAPI, PostgreSQL.

List of Figures

3.1 Overall system architecture (block diagram) . . . 12
3.2 DFD level 0 (context diagram) . . . 13
3.3 DFD level 1 . . . 13
3.4 Use case diagram . . . 14
3.5 Simplified class diagram of the core domain . . . 15
3.6 Sequence diagram for a natural-language search . . . 15
3.7 ER diagram (main entities; personal sections, notifications, tokens and audit omitted) . . . 16
4.1 Flowchart of the extraction pipeline . . . 18
4.2 Flowchart of the query engine . . . 22
5.1 Input: page 1 of the college's real timetable PDF (S.E. Division A), with stacked lab cells and legend . . . 25
5.2 Student screens . . . 25
5.3 Natural-language search (Ask tab) . . . 26
5.4 Combined classes, HOD view, conflicts and personal domain . . . 26
5.5 Administrator workflow . . . 27
5.6 Composed query plans answered from the real timetable (TE-B4 student) . . . 28
5.7 The same app in a laptop browser . . . 29
5.8 Effect of intended-overlap rules: from 542 raw overlaps to the 2 genuine clashes . 30
5.9 Throughput and latency under increasing concurrent load (measured) . . . 31

List of Tables

1.1 Technologies used 3
2.1 Comparison of existing approaches with the proposed system 8
3.1 Functional requirements 9
3.2 Non-functional requirements 11
4.1 Threats and controls 23
5.1 Contribution of group members 24
5.2 Extraction performance on the real 10-page PDF (median of 5 runs) 29
5.3 Single-user search latency per question type (50 requests each) 30
5.4 Load test with increasing concurrent clients (4 API workers) 31
5.5 Test case specification 31
5.6 Objectives mapped to evidence 34
5.7 Improvements incorporated before the ESE 35
5.8 Scenario-based challenges and how the system responds 35

Contents

Abstract iii

List of Figures iv

List of Tables v

1 Introduction 1
- 1.1 Problem Statement . . . 1
- 1.2 Objectives . . . 2
- 1.3 Scope . . . 2
- 1.4 Technologies Used . . . 3
- 1.5 Assumptions and Constraints . . . 3

2 Literature Survey 5
- 2.1 Literature Survey 1: A Survey of Automated Timetabling . . . 5
- 2.2 Literature Survey 2: Recent Research Directions in Automated Timetabling . . . 5
- 2.3 Literature Survey 3: Meta-heuristics for University Timetabling . . . 5
- 2.4 Literature Survey 4: A Survey of Table Recognition . . . 5
- 2.5 Literature Survey 5: The T-Recs Table Recognition System . . . 6
- 2.6 Literature Survey 6: ICDAR 2013 Table Competition . . . 6
- 2.7 Literature Survey 7: An Overview of the Tesseract OCR Engine . . . 6
- 2.8 Literature Survey 8: Natural Language Interfaces to Databases . . . 6
- 2.9 Literature Survey 9: Seq2SQL . . . 6
- 2.10 Literature Survey 10: The Spider Text-to-SQL Benchmark . . . 6
- 2.11 Literature Survey 11: Indirect Prompt Injection . . . 7
- 2.12 Literature Survey 12: From Prompt Injection to SQL Injection . . . 7
- 2.13 Literature Survey 13: Maintaining Knowledge about Temporal Intervals . . . 7
- 2.14 Literature Survey 14: Role-Based Access Control Models . . . 7
- 2.15 Literature Survey 15: Argon2 Memory-Hard Hashing . . . 7
- 2.16 Comparison with Existing Systems . . . 7
- 2.17 Research Gap . . . 8

3 Analysis 9
- 3.1 Requirements Analysis . . . 9
- 3.2 System Architecture . . . 11
- 3.3 Data Flow Diagrams . . . 12
- 3.4 Use Case Diagram . . . 14
- 3.5 Class Diagram . . . 14

3.6 Sequence Diagram 15
3.7 ER Diagram 16

4 Design and Methodology 17
4.1 Proposed System 17
4.2 Dataset 18
4.3 Time-Row Resolution by Constrained Backtracking 19
4.4 Cell Parsing 19
4.5 Conflict Detection (Group, Sort and Sweep) 20
4.6 Compositional Query Planner 20
4.7 Security Design 22

5 Results and Discussion 24
5.1 Implementation 24
5.2 Screenshots 24
5.3 Extraction Results 29
5.4 Clash Detection Results 29
5.5 Query Results and Performance 30
5.6 Testing 31
5.7 Comparative Analysis 34
5.8 Improvements Incorporated after Review 34
5.9 Scenario-Based Challenges 35
5.10 Discussion 37

6 Conclusion and Further Work 38
6.1 Conclusion 38
6.2 Limitations 38
6.3 Further Work 38

7 Research Paper/ Poster 40
7.1 Research Paper 40
7.2 Poster 47

8 Plagiarism Report 49
8.1 Plagiarism Report 49
8.2 AI Content Detection Report 49

References 50

Chapter 1: Introduction

Every semester the timetable committee of the college publishes the class timetable as a multi-page PDF: one grid per division for S.E., T.E., B.E. and M.Tech., with days as rows, time slots as columns and, below each grid, a legend that expands abbreviations such as `DBMS`, `KKD` or `LLC`. A single cell may contain a subject, a teacher code, a room number and, for practical sessions, four stacked lab batches (`A1`–`A4`), each with its own teacher and laboratory.

Students and teachers read this document by hand many times a day: to find the next lecture, to see whether a lab is in 606 or 607, to locate a teacher, or to find an empty room for a make-up lecture. When a revised PDF is circulated, everyone compares it with the old one by eye, and clashes (one teacher in two rooms at the same time) are noticed only when they happen. *Dynamic Timetable* reads the timetable exactly as it is published, checks it, and lets every student and teacher see their schedule and ask questions about it in plain English on a phone or a laptop.

1.1 Problem Statement

> *Design and implement a system that converts the college's published timetable (PDF, spreadsheet, Word file or photograph) into verified structured data, keeps every version as an auditable archive, detects scheduling clashes, and lets each user (student, professor, HOD, Principal or administrator) get correct answers to everyday timetable questions on a phone or laptop, including questions typed in plain English, without exposing one user's personal data to another.*

The difficulties with the current practice are:

1. **Unstructured source.** Merged cells, stacked lab rows, page-wise legends, inconsistent time labels (the noon row is printed "12.15 a.m. to 01.15 p.m.") and scanned pages make automatic reading hard.
2. **Manual lookup.** "Where is my next class?" requires finding the right page, row, column and lab batch and then expanding the legend codes.
3. **No conflict visibility.** Teacher, room and batch overlaps are not checked mechanically.
4. **Version confusion.** Old and revised PDFs circulate together.
5. **Privacy.** A personal timetable (coaching, tuition) must never be visible to others.

1.2 Objectives

O1. Extract classes from the real timetable PDF with at least 95% verified automatically, and flag every uncertain value for review instead of guessing.

O2. Accept PDF, scanned PDF or photo (OCR), Excel/CSV and Word through one pipeline.

O3. Provide role-based views for Student, Professor, HOD, Principal and Admin, with permissions enforced on the server.

O4. Answer everyday questions typed in English with a deterministic engine that is not limited to a fixed list of questions, and ask for clarification when a question is ambiguous.

O5. Detect professor, room and batch clashes and report only genuine clashes.

O6. Keep official and personal timetables strictly separate, with full version history.

O7. Answer a question in under 300 ms (median) with no wrong answers under concurrent use.

O8. Run on Android, iPhone and laptop browsers from one code base, with a free online demo.

O9. Back the system with automated tests, all passing.

1.3 Scope

Included. Upload and extraction (PDF, scanned PDF, JPG/PNG, XLSX/XLS, CSV, DOCX/-DOC); validation, review and audited correction; an archive of all versions with one official timetable; clash detection; role dashboards; weekly timetable of any batch, teacher or room; natural-language questions; free-room and teacher-availability search; personal timetables; year-end promotion of students; administrator master data; audit log.

Not included. Automatic *generation* of a timetable (the committee still creates it); attendance; live location of teachers (only the *scheduled* room is reported); exam timetables; integration with the college ERP.

Users and scale. Students, professors, HODs, the Principal and the timetable administrator of one institute: about 2,000–3,000 users and 500–1,000 classes per timetable.

1.4 Technologies Used

Table 1.1: Technologies used

Layer	Technology	Purpose
Back end	Python 3.13, FastAPI [31]	Typed REST API with automatic OpenAPI documentation
Database	PostgreSQL 16 [30], SQLAlchemy 2, Alembic	31 normalised tables, migrations; also the job queue (`SKIP LOCKED`)
Extraction	PyMuPDF [29], Tesseract OCR [7], openpyxl, python-docx	Reading PDF grids, scanned pages, Excel and Word files
App	React Native [33], Expo SDK 58 [32], TypeScript, TanStack Query	One code base for Android, iOS and the web
Security	Argon2id [15], JWT [18], RBAC [14]	Password hashing, rotating tokens, role-based access
Testing	pytest, Jest, Playwright	Unit, integration and end-to-end tests
Deployment	Docker, Render [34]	Free cloud demo from one blueprint file

1.5 Assumptions and Constraints

Assumptions.

- The timetable repeats weekly within a semester (date-specific entries are supported but rare).
- Every user has a college e-mail address issued by the institute.
- Room floors can be derived from the room number (508 $\rightarrow$ floor 5) until the Admin corrects them.
- A class from 9:00 to 10:00 occupies the interval $[9{:}00, 10{:}00)$, so back-to-back classes do not clash.

Constraints.

- **Financial:** zero budget, so only open-source software and free hosting were used.
- **Technical:** the source is a human-formatted PDF whose layout can change every semester; the parser must report what it cannot read.

- **Operational:** the official timetable changes only after explicit Admin activation, and the Admin can never read personal timetables.
- **Time:** one semester alongside regular course work.

Chapter 2: Literature Survey

We studied fifteen research papers in four areas that the project draws on: automated timetabling, table recognition and OCR, natural-language interfaces to databases (NLIDB), and temporal reasoning and security. Each is summarised below with what it contributes and what it leaves open for our problem. Section 2.17 states the gap that our system fills.

2.1 Literature Survey 1: A Survey of Automated Timetabling

Schaerf [1] classifies school, course and examination timetabling problems and the search techniques used to solve them (graph colouring, integer programming, local search). **Relevance:** it defines the hard clash constraints (no teacher, room or student group in two places at once) that we *check*. **Gap:** it assumes the timetable is generated by software; reading an already published timetable is not addressed.

2.2 Literature Survey 2: Recent Research Directions in Automated Timetabling

Burke and Petrovic [2] review constraint-based, meta-heuristic and hybrid approaches for university timetabling and point out the distance between research prototypes and real institutional data. **Relevance:** confirms that real data is messy and must drive the design. **Gap:** no support for the consumers of a timetable after it is published.

2.3 Literature Survey 3: Meta-heuristics for University Timetabling

Lewis [3] surveys meta-heuristic techniques and separates hard constraints from soft ones (preferences, compactness). **Relevance:** the hard constraints become our conflict rules. **Gap:** construction only; no search, role views or clash checking of a published grid.

2.4 Literature Survey 4: A Survey of Table Recognition

Zanibbi, Blostein and Cordy [4] describe table recognition as detection, structure recognition (rows, columns, spanning cells) and interpretation, and note that interpretation is domain specific. **Relevance:** motivated our timetable-specific grid parser. **Gap:** no interpretation of lab batches, legends or time rows.

2.5 Literature Survey 5: The T-Recs Table Recognition System

Kieninger and Dengel [5] recognise tables from word bounding boxes without relying on ruling lines. **Relevance:** basis of our geometric fallback that rebuilds columns from whitespace gutters when a page has no ruled lines. **Gap:** generic tables only.

2.6 Literature Survey 6: ICDAR 2013 Table Competition

Göbel et al. [6] define standard metrics for table location and structure recognition and show that cell-structure recovery remains error-prone. **Relevance:** led us to give every extracted entry a verification status instead of trusting the extractor. **Gap:** evaluation of extraction, not of downstream use.

2.7 Literature Survey 7: An Overview of the Tesseract OCR Engine

Smith [7] describes Tesseract's line finding, adaptive classifier and word recognition. **Relevance:** we use Tesseract for scanned pages and photographs. **Gap:** OCR output has no table semantics, so OCR entries are capped at UNVERIFIED in our system.

2.8 Literature Survey 8: Natural Language Interfaces to Databases

Androutsopoulos, Ritchie and Thanisch [8] review pattern-matching, syntax-based and semantic-grammar NLIDB systems and observe that a narrow domain allows simple, reliable rule-based interpretation. **Relevance:** supports our deterministic planner. **Gap:** classical systems map a question to one fixed pattern; we compose independent pieces into a plan.

2.9 Literature Survey 9: Seq2SQL

Zhong, Xiong and Socher [9] generate SQL from text with a neural sequence model trained by reinforcement learning. **Relevance:** shows the flexibility of learned text-to-SQL. **Gap:** needs training data and is not exact; a wrong query sends a student to the wrong room.

2.10 Literature Survey 10: The Spider Text-to-SQL Benchmark

Yu et al. [10] introduce a large cross-domain text-to-SQL benchmark; exact-match accuracy on complex queries is far below 100%. **Relevance:** quantifies the accuracy risk of generated SQL. **Gap:** no guarantee of correctness or safety for a single deployed domain.

2.11 Literature Survey 11: Indirect Prompt Injection

Greshake et al. [11] show that LLM-integrated applications can be taken over by instructions hidden in the data they read. **Relevance:** letting an LLM drive data access is a security risk. **Gap:** mitigation remains an open problem.

2.12 Literature Survey 12: From Prompt Injection to SQL Injection

Pedro et al. [12] demonstrate prompts that make LLM web applications run malicious SQL. **Relevance:** the main reason our engine never builds SQL from user text and uses only pre-written parameterised templates.

2.13 Literature Survey 13: Maintaining Knowledge about Temporal Intervals

Allen [13] defines thirteen relations between time intervals. **Relevance:** our clash rule uses the overlap family $a.s < b.e \wedge b.s < a.e$ with end-exclusive intervals.

2.14 Literature Survey 14: Role-Based Access Control Models

Sandhu et al. [14] formalise RBAC with roles, permissions and sessions. **Relevance:** our five roles and their data scopes, enforced on the server for every request.

2.15 Literature Survey 15: Argon2 Memory-Hard Hashing

Biryukov, Dinu and Khovratovich [15] present Argon2, the winner of the Password Hashing Competition, which resists GPU cracking. **Relevance:** used for all stored passwords.

2.16 Comparison with Existing Systems

Besides the papers, we compared the tools a college could use today (Table 2.1): timetable generators (FET, UniTime, aSc [24–26]), PDF table extractors (Tabula, Camelot [27, 28]), calendar apps, ERP modules and LLM text-to-SQL.

Table 2.1: Comparison of existing approaches with the proposed system

Criterion	Manual PDF	FET / UniTime / aSc	Tabula / Camelot	Calendar apps	ERP module	LLM text-to-SQL	Proposed
Reads published PDF/photo	–	No	Partly	No	No	No	**Yes**
Timetable semantics (labs, legends)	Human	Yes	No	No	Yes	Depends	**Yes**
Role-based views	No	Limited	No	No	Yes	No	**Yes (5)**
Plain-English questions	No	No	No	Limited	No	Yes	**Yes (composed plans)**
Reproducible, injection-safe answers	–	–	–	–	Yes	No	**Yes**
Clash detection	No	At creation	No	No	Sometimes	No	**Yes, after publication**
Version archive	Ad hoc	Files	No	No	Sometimes	No	**Yes, audited**
Personal timetable kept private	–	No	No	Yes	No	No	**Yes**
Cost	Free	Free/Paid	Free	Free	Paid	Paid API	**Free (open source)**
Mobile use	PDF viewer	Limited	No	Yes	Varies	Varies	**Android, iOS, web**

2.17 Research Gap

1. Timetabling research and generators *construct* timetables; none reads the timetable as it is published or helps the people who follow it [1–3].
2. Table recognition recovers cells but not timetable meaning: stacked lab batches, page-wise legends and mislabelled time rows [4, 6].
3. Text-to-SQL and LLM interfaces are flexible but not exact and are open to injection [10–12].
4. No tool used by our college checks the *published* timetable for clashes.

The gap is therefore not a better construction algorithm, but the missing link between the published document and its daily users: trustworthy, role-scoped, searchable data with provenance, clash checks and version history, and safe answers to plain-English questions.

Chapter 3: Analysis

This chapter analyses what the system must do and how its parts and data relate: the requirements, the overall architecture, data flow diagrams and UML diagrams.

3.1 Requirements Analysis

The functional requirements were derived from the problems in Section 1.1 and from discussions with students, faculty and the guide. Table 3.1 lists them with their acceptance criteria and Table 3.2 the non-functional targets.

Table 3.1: Functional requirements

ID	Requirement	Input → processing → output	Acceptance criterion
FR-01	Sign-in with college e-mail and password	E-mail, password → Argon2id verify, account-status check, rate limit → 15-min access JWT + rotating refresh token	Wrong e-mail or password gives the same generic error; suspended account refused
FR-02	Five roles enforced on the server	Request + token → roles loaded from `USER_ROLE` on every request → allowed / 403	No endpoint trusts a client-sent role; role change invalidates old tokens
FR-03	Upload a timetable (PDF, image, XLSX/XLS, CSV, DOCX/DOC)	File → extension allow-list, content-signature check, size/page limits, private storage, SHA-256, job queued → archive record (QUEUED)	Upload never changes the primary timetable; retries with the same request id are idempotent
FR-04	Extract class entries	Stored file → worker: page inspection, table/OCR extraction, sections, time rows, cell parsing, legend expansion, mapping → entries with provenance and status	Real college PDF: $\geq$ 95% of class entries VERIFIED; nothing guessed silently
FR-05	Validate and classify the timetable	Entries → checks (missing fields, duplicates, time consistency, overlaps) → READY / NEEDS REVIEW / UNUSABLE / FAILED	Unusable timetables cannot become primary
FR-06	Review and correct entries	Admin edits → before/after snapshot in `ENTRY_CORRECTION`, audit row → corrected entry	Original value always recoverable; entry becomes VERIFIED only by explicit reviewer action

ID	Requirement	Input → processing → output	Acceptance criterion
FR-07	Activate the official timetable	Admin action → settings row locked, primary switched, version incremented, users notified → new primary	Concurrent activations serialise; next request sees the new primary
FR-08	Archives and selection	User picks an older version → selection remembered per domain → searches run on that archive with a banner	Archive browsing never mixes domains
FR-09	Role dashboard	Login → current class, next class with countdown, free periods, later today → cards	Dashboard and typed questions give identical answers
FR-10	Weekly timetable view	Day selection → entries for the user's batch and lab group (and parent division) → list with room/teacher	Lab group A1 does not see A2's labs
FR-11	Natural-language search	English text → parse, build a query plan (operation, filters, lab/lecture, date/week, time window, sort), resolve entities in this timetable, validate, compile with role scope → result envelope	No SQL generated from text; counts, first/last and day-wise questions answered; ambiguous a.m./p.m., date or entity asks for clarification
FR-12	Free-room and professor-availability search	Time/date, optional floor → room inventory minus scheduled rooms → free rooms / free periods	Uncertain entries treated as busy; incomplete inventory reported
FR-13	Conflict detection	Timetable → interval overlap per professor, room, batch, excluding parallel lab groups and electives → conflict list	Real PDF: exactly the 2 genuine clashes reported
FR-14	Personal timetables	Own upload → same pipeline, stored in personal tables → private timetable	Other users (including Admin) get 404 for it
FR-15	Academic promotion	Admin preview + confirm → transactional update of placements with history → students moved to next year/-batch	All-or-nothing; history preserved; concurrent promotions cannot double-apply
FR-16	Master data management	Admin forms → validated CRUD of departments, years, batches, courses, rooms, aliases → records	Duplicates and invalid references rejected
FR-17	User and account management	Admin → register student/staff, roles, suspend, issue access code → accounts	Suspension revokes sessions immediately

ID	Requirement	Input → processing → output	Acceptance criterion
FR-18	Notifications	Activation and job completion → in-app notification (push in store builds) → notification list	Users see only their own notifications
FR-19	Audit log	Sensitive actions → append-only audit events → Admin audit screen	Uploads, activations, corrections, promotions, access denials logged

Table 3.2: Non-functional requirements

ID	Category	Requirement
NFR-01	Performance	Median search response < 300 ms for one user; extraction of a 10-page PDF < 10 s.
NFR-02	Scalability	Stateless API that scales horizontally by adding processes; workers scale independently.
NFR-03	Availability, reliability	Correct answers under concurrent use; idempotent uploads; failed jobs retried up to 3 times; health endpoint for the platform.
NFR-04	Security	Argon2id passwords, JWT + refresh-token rotation and reuse detection, RBAC, IDOR protection, rate limiting, parameterised SQL only, secrets never in the app bundle.
NFR-05	Privacy	Personal timetables visible only to their owner; no free text in logs.
NFR-06	Usability, accessibility	Readable in light and dark mode; large tap targets; plain-English questions; clarifications with tappable choices.
NFR-07	Maintainability	Modular layers (API, services, query, extraction); typed code (Pydantic, TypeScript strict); automated tests; documented decisions.
NFR-08	Portability	One code base for Android, iOS and web; Docker image for any Linux host.
NFR-09	Resource utilisation	One API worker < 300 MB RAM; database < 50 MB for one institute's timetables.
NFR-10	Auditability	Every entry keeps raw text, page, parser version and status; every correction keeps before/after.

3.2 System Architecture

Figure 3.1 shows the architecture. One React Native app serves Android, iOS and the web. It talks over HTTPS to a stateless FastAPI back end that authenticates every request, loads the user's roles from the database and resolves the search context on the server. Uploads are

queued in PostgreSQL and processed by a background worker, so the API is never blocked by OCR. PostgreSQL is the single source of truth.

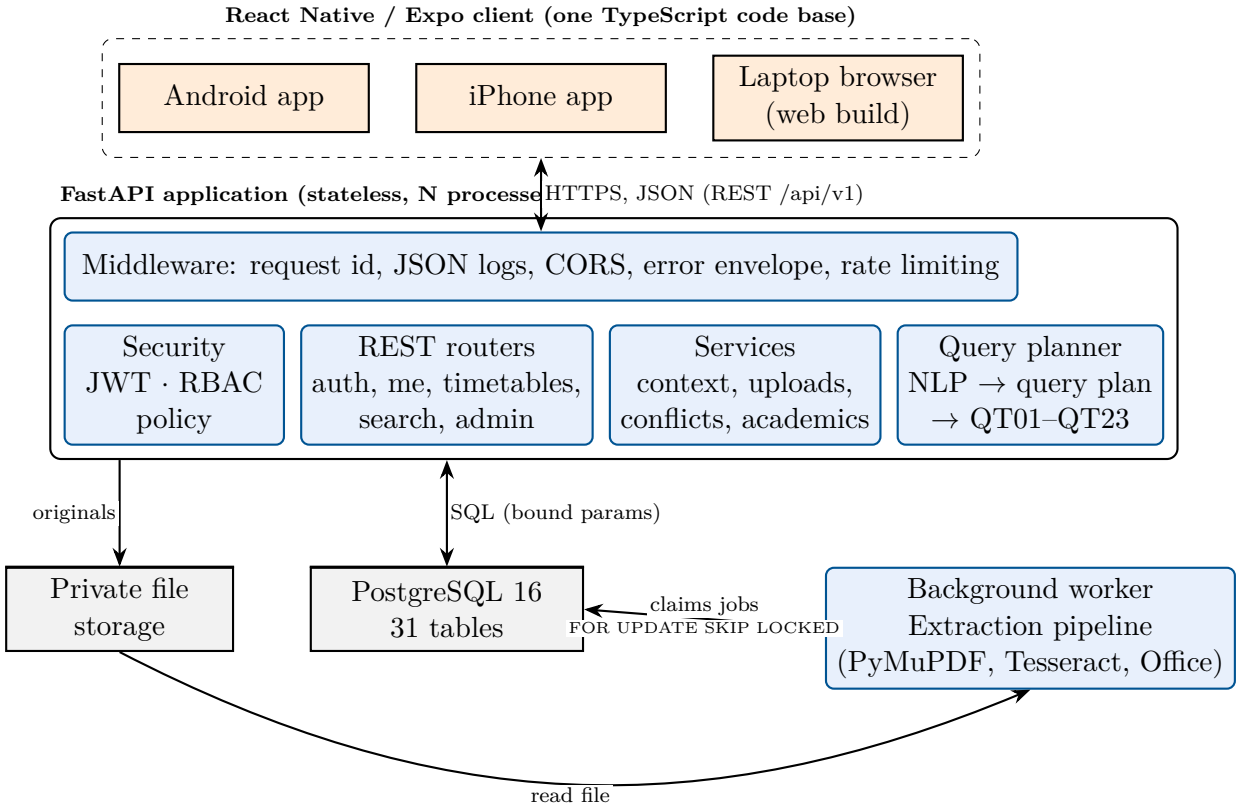


Figure 3.1: Overall system architecture (block diagram)

3.3 Data Flow Diagrams

The context diagram (Figure 3.2) shows the system and its external entities. Level 1 (Figure 3.3) splits it into the main processes: authentication, upload and extraction, review and activation, conflict detection and question answering, all sharing the timetable store.

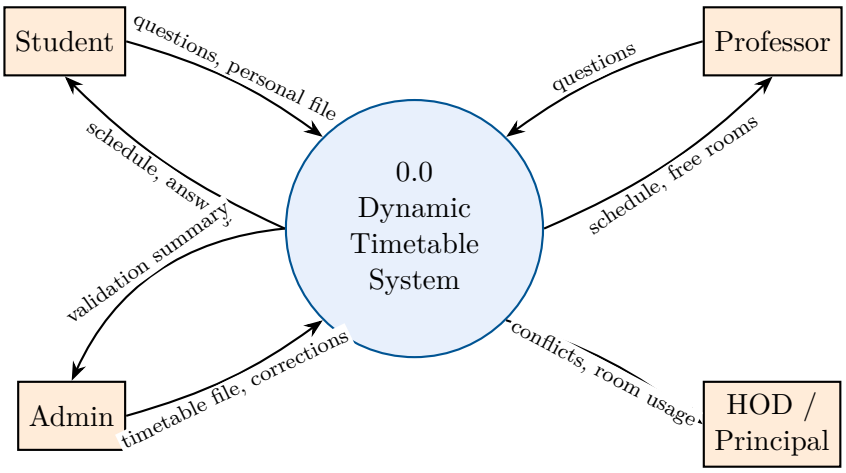


Figure 3.2: DFD level 0 (context diagram)

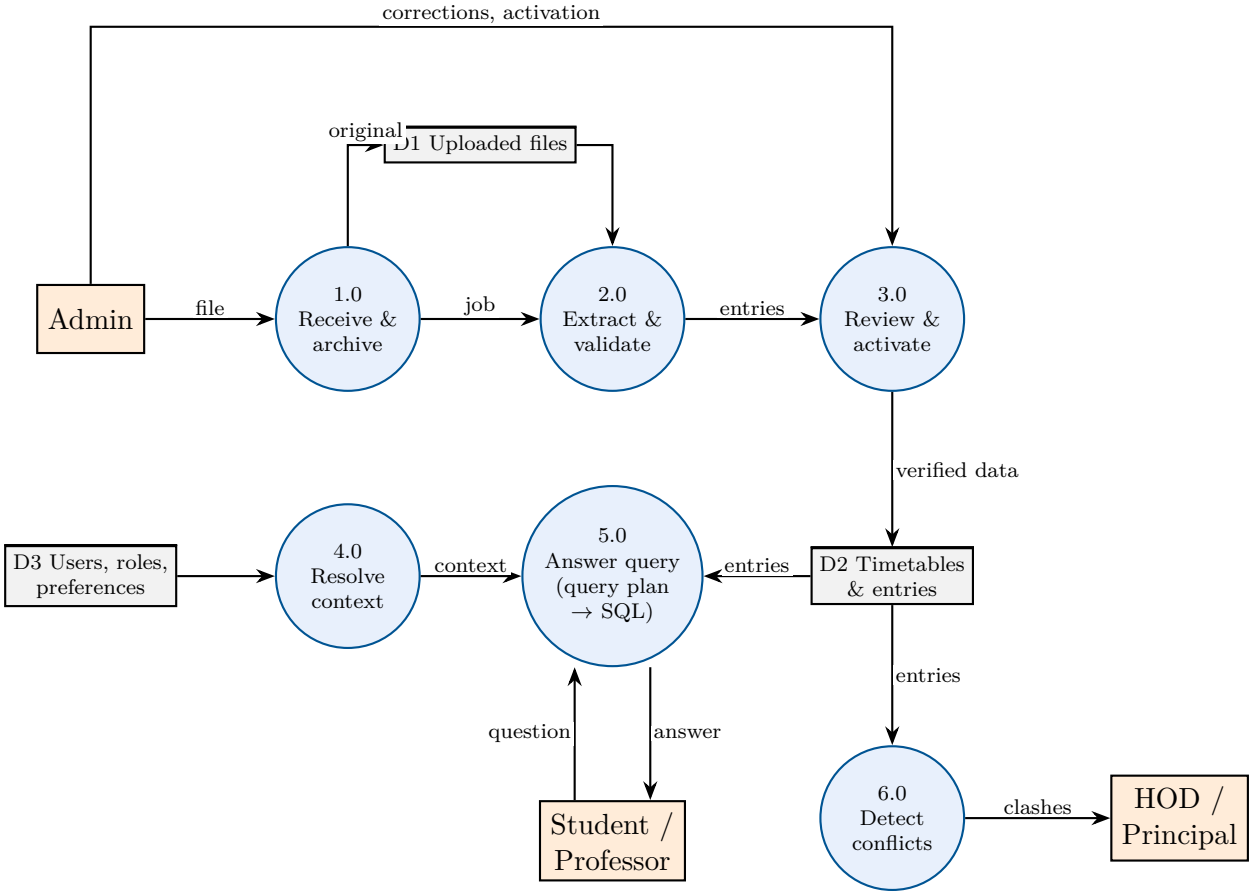


Figure 3.3: DFD level 1

3.4 Use Case Diagram

Figure 3.4 shows what each role can do. Every logged-in user can view any official timetable and ask questions; personal timetables belong only to their owner; clash reports are for the HOD, Principal and Admin; upload, correction and activation are Admin-only.

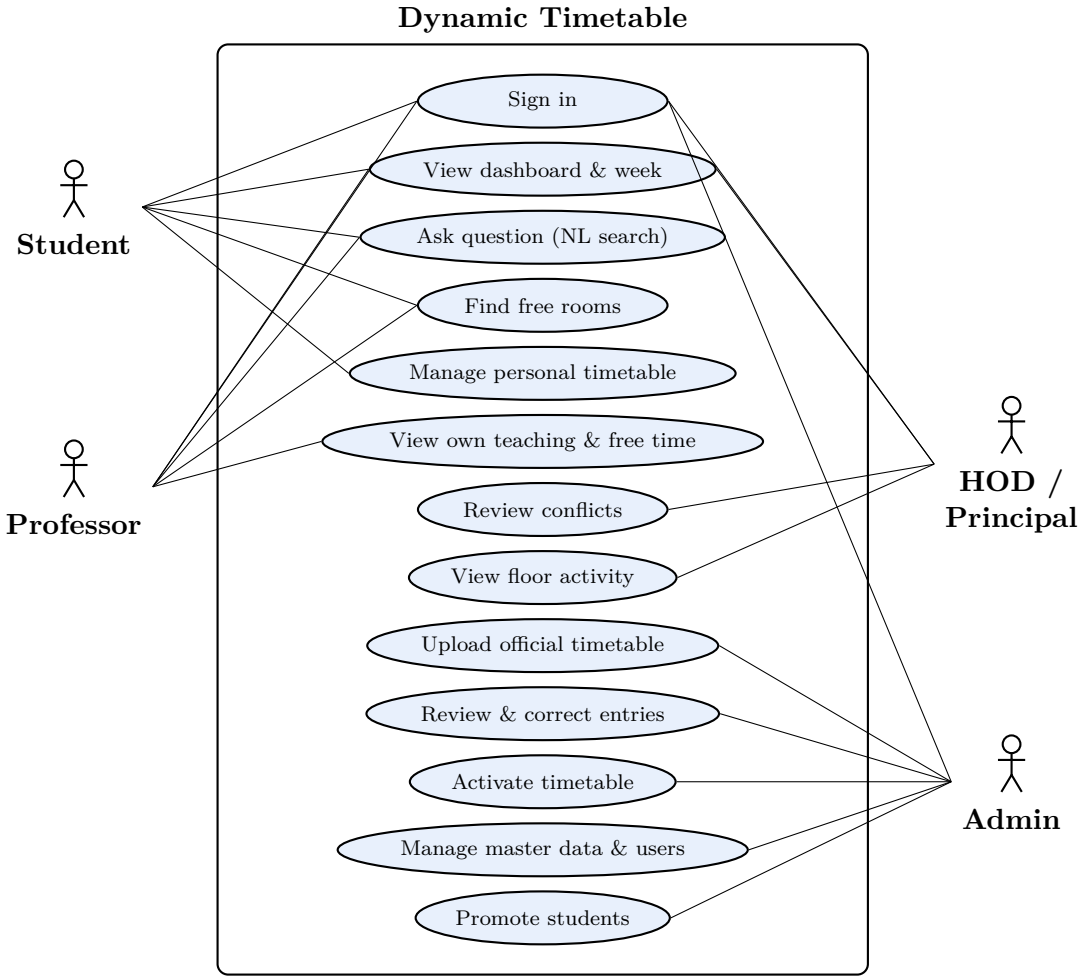


Figure 3.4: Use case diagram

3.5 Class Diagram

Figure 3.5 shows the core domain: a timetable has many entries, and each entry refers to a course, a teacher (staff), a room and a batch; batches form a hierarchy (division → lab batch).

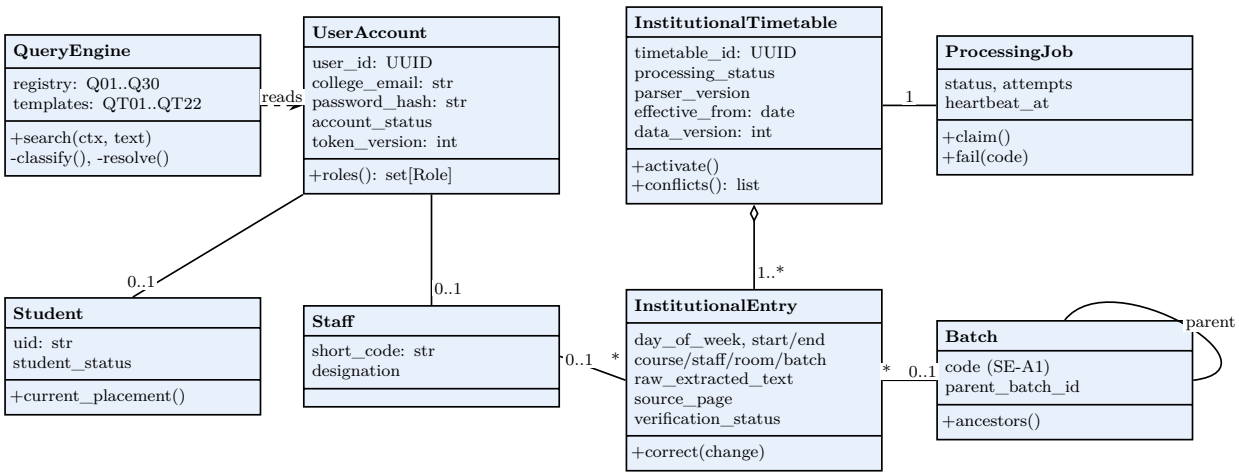


Figure 3.5: Simplified class diagram of the core domain

3.6 Sequence Diagram

Figure 3.6 follows one question from the app to the database and back: authentication, context resolution, planning, validation and execution of a parameterised query.

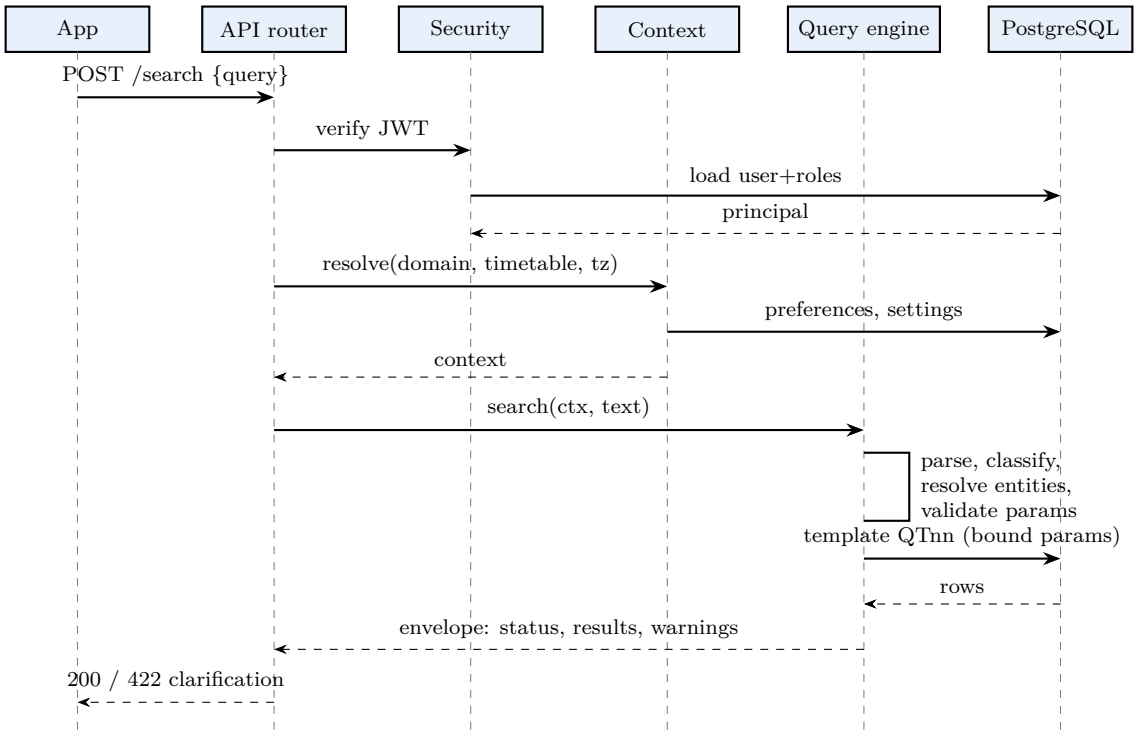


Figure 3.6: Sequence diagram for a natural-language search

3.7 ER Diagram

Figure 3.7 shows the main entities of the database. The schema has 31 tables in third normal form; course, staff, room and batch details are stored once and referenced by foreign keys.

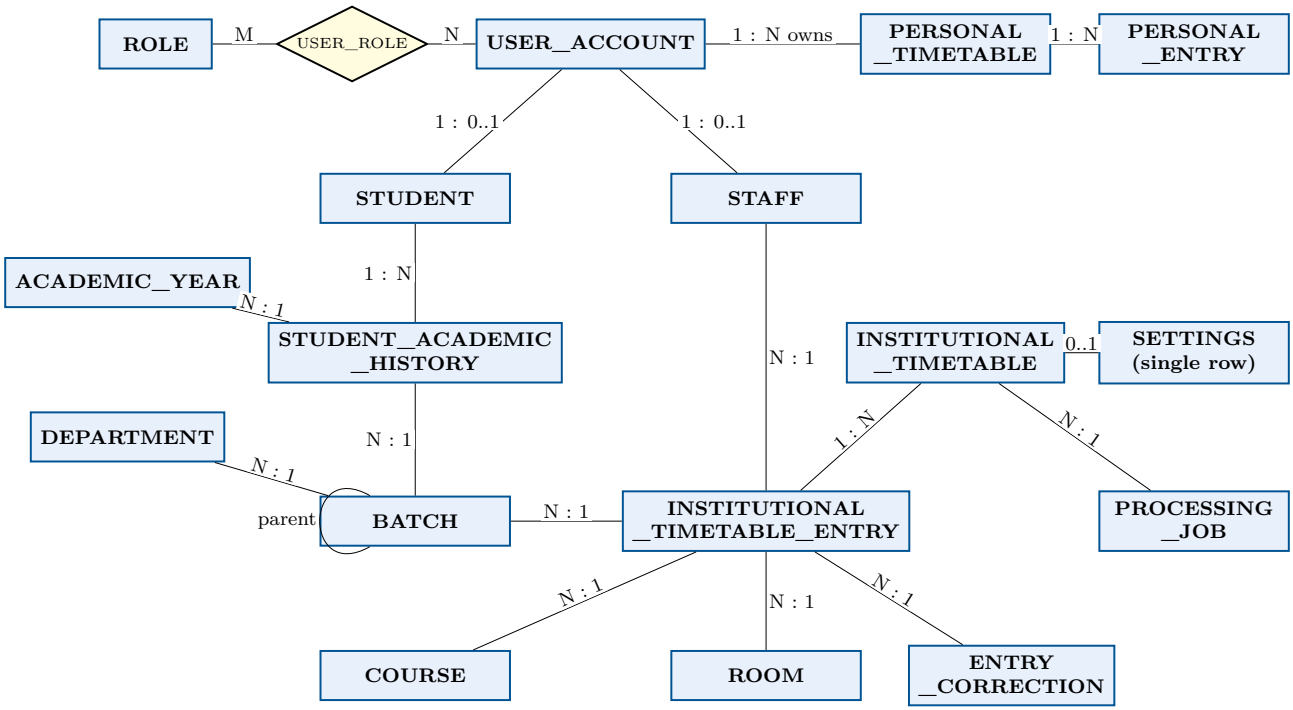


Figure 3.7: ER diagram (main entities; personal sections, notifications, tokens and audit omitted)

Chapter 4: Design and Methodology

4.1 Proposed System

The proposed system has three major modules (Figure 4.1 and Figure 4.2):

1. **Extraction pipeline.** Converts any supported file into a common grid model, recognises sections, time rows, lab batches, legends and breaks, validates every entry and gives it a status: Verified, Unverified, Incomplete or Rejected.
2. **Back end.** Stores each timetable as an archive, enforces role-based access, detects clashes and answers questions through a deterministic *compositional query planner*.
3. **Cross-platform app.** Role dashboards, weekly view of any batch, teacher or room, the Ask screen, archives, uploads, review screens and administrator tools.

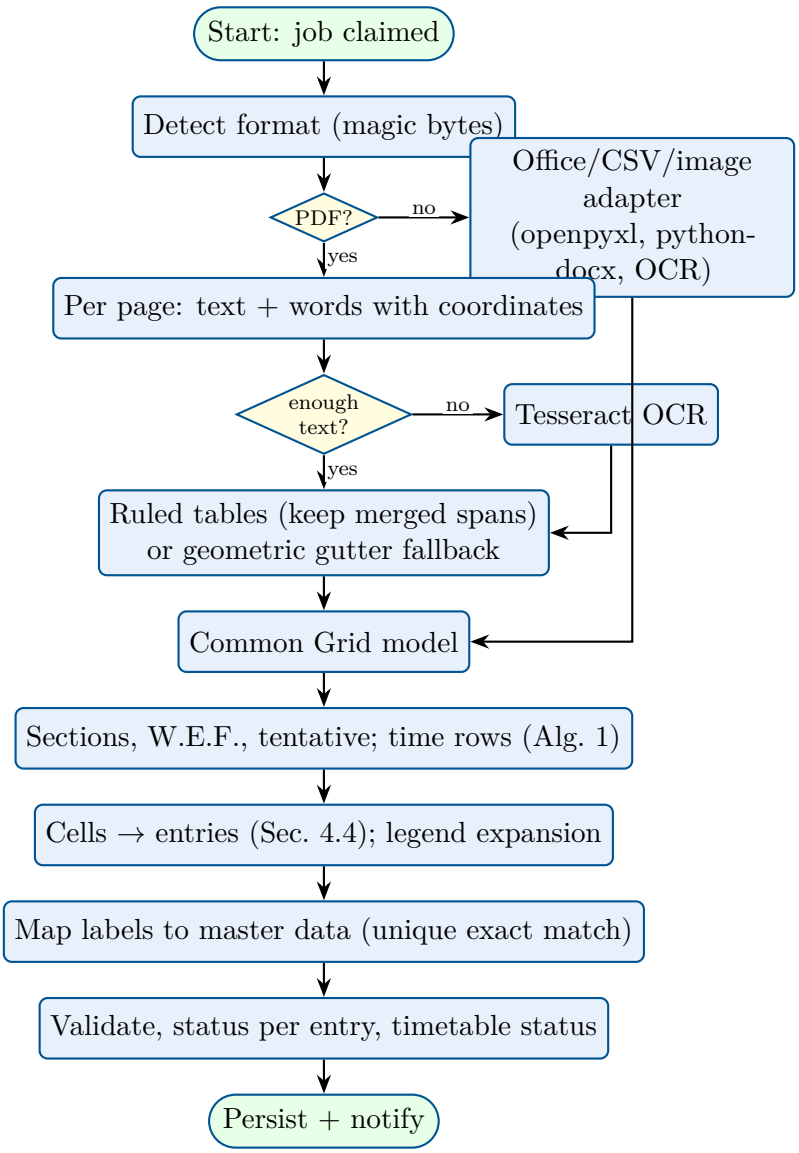


Figure 4.1: Flowchart of the extraction pipeline

4.2 Dataset

The system was developed and evaluated on the Computer Engineering department's real timetable for the current semester: a 10-page PDF covering S.E., T.E., B.E. and M.Tech. divisions with 453 classes (570 rows including breaks and activities). Problems found in this document shaped the design: lab cells with four batches, codes expanded only in each page's legend, the noon row printed "12.15 a.m. to 01.15 p.m.", a combined B.E. elective page, LLC slots with no teacher or room, and spelling variants ("Sushama"/"Sushma" Pande, "Discreate" Structures). A smaller reference PDF and synthetic Excel, Word and scanned files were used for the other input formats.

4.3 Time-Row Resolution by Constrained Backtracking

Each time label is turned into a small set of candidate intervals. The algorithm searches for assignments in which every row is plausible (between 7:00 and 21:00, at most 6 hours) and no row starts before the previous row ends. A reading is accepted as certain only if it is the *same in every valid assignment*; otherwise the row goes to review. The college's noon row is therefore read as 12:15–13:15 only because the row before ends at 12:15 and the row after starts at 13:15. The search is bounded at 64 assignments, so it costs $O(64r)$ for r rows.

Algorithm 1 Resolve a column of time labels

Input: labels $L_1..L_r$ (top to bottom); each L_i gives start/end hour, minute and optional marker
Output: for each row: (start, end, certain?) or a review flag

for $i \leftarrow 1$ **to** r **do**
  $O_i \leftarrow \{(s,e) : s \in \textsc{Cand}(L_i.start), e \in \textsc{Cand}(L_i.end), \textsc{Plausible}(s,e)\}$ ▷ literal markers honoured
  if $O_i = \emptyset$ **then** $O_i \leftarrow$ readings ignoring markers, flagged *non-literal*
  end if
end for
$S \leftarrow \emptyset$
procedure $\textsc{Walk}(i, prevEnd, A)$
  if $|S| > 64$ **then return**
  end if ▷ bound the search
  if $i > r$ **then** $S \leftarrow S \cup \{A\}$; **return**
  end if
  for all $(s,e) \in O_i$ **with** $s \geq prevEnd$ **do**
    $\textsc{Walk}(i+1, e, A \cdot (s,e))$
  end for
end procedure
$\textsc{Walk}(1, 0, \langle\rangle)$
for $i \leftarrow 1$ **to** r **do**
  $R_i \leftarrow \{A_i : A \in S\}$
  if $|R_i| = 1$ **and** reading is literal **then** mark row i CERTAIN
  else if $|R_i| = 1$ **then** keep reading, set `time_uncertain`, add `TIME_LABEL_INCONSISTENT`
  else leave time empty, add `TIME_AMBIGUOUS_AMPM` (entries become INCOMPLETE)
  end if
end for

4.4 Cell Parsing

A cell is split into lines, lines belonging to one lab batch (e.g. `A1 COA Lab JS 606-4`) are grouped, and the batch (`[A-D][1-4]`), room, teacher codes and subject are identified. Codes

are expanded only if they appear in the legend of the *same page*; otherwise the code is kept and an informational flag is raised.

4.5 Conflict Detection (Group, Sort and Sweep)

Entries are grouped by resource (teacher, room, batch) and day, sorted by start time and swept forward; the inner loop stops as soon as a later entry starts after the current one ends, giving $O(n \log n + m)$ instead of $O(n^2)$ [21]. Intended overlaps (the same source cell, parallel lab batches, elective options, a combined session printed on several pages) are not reported. The overlap test follows Allen's interval relations [13].

Algorithm 2 Detect professor, room and batch clashes

Input: entries E of one timetable (classes with day and times, not rejected)
Output: list of conflicts with confidence CONFIRMED/POSSIBLE

$G \leftarrow$ empty hash map
for all $x \in E$ **do**
  for all resource $(k, key) \in$ {professor, room, batch} of x **do**
    append x to $G[(k, key, x.day)]$ ▷ id if mapped, else normalised label
  end for
end for
for all group $g \in G$ **do**
  sort g by start time
  for $i \leftarrow 1$ **to** $|g|$ **do**
    for $j \leftarrow i + 1$ **to** $|g|$ **do**
      if $g_j.start \geq g_i.end$ **then break** ▷ sorted: no later entry can overlap
      end if
      if $g_i.start < g_j.end \wedge g_j.start < g_i.end$ **and not** INTENDED(g_i, g_j) **then**
        report (k, g_i, g_j); CONFIRMED if both verified, mapped and certain
      end if
    end for
  end for
end for
function INTENDED(a, b): same source cell, same combined session on several pages, parallel elective options, or a sub-group's own written period
end function

4.6 Compositional Query Planner

The engine does not match a whole question to one of a fixed list. It recognises independent pieces and combines them into a typed **query plan**:

- **Operation** (17): LIST, COUNT, EXISTS, TOTAL_HOURS, FIRST, LAST, NEXT,

Per_Day, Span, Free_Slots, Breaks, After/Before_Course, Teachers, Per_Room, Room_Info, Conflicts.

- **Filters**: me, batch, professor, room, course; lab or lecture; room type.
- **When**: a date, a weekday, this or next week, or the whole week.
- **Time window**: at, after, before, between, after lunch, now.
- **Metric and sort**: most/least classes or hours, earliest/latest start or finish.

For example, "Which day do I finish earliest?" becomes `{operation: PER_DAY, metric: LAST_END, sort: EARLIEST}`. Names are matched against a vocabulary built from the timetable itself: a teacher can be named by code ("AVN"), first name, surname or "Nimkar sir"; entries sharing a faculty code are merged; subjects match partial words and one-letter typos. The plan is validated against a closed registry and compiled into pre-written parameterised SQL, so user text never becomes SQL (safe against the attacks in [11, 12]). Ambiguous questions ("at 2", a surname shared by two teachers) get a clarification instead of a guess, and every answer carries its plan.

Algorithm 3 Build a query plan from the parsed question

Input: parsed question p, specialised operation S recognised by the rule cascade (or $\perp$)
Output: plan P

1: **if** S is pinned (archives, primary, cross-domain, time-until) **then return** $P(S)$
2: **end if**
3: $k \leftarrow$ Lab / Lecture from words ("lab", "practical" / "lecture", "theory"); $\perp$ otherwise
4: $w \leftarrow$ window from times and words (between, before, after, after lunch, at)
5: $o \leftarrow$ Total_Hours ("total hours") $\vee$ Count ("how many") $\vee$ Per_Day ("which day", "busiest") $\vee$ Next / First / Last $\vee$ Exists ("any lab")
6: **if** $o =$ Per_Day **then** choose metric (finish $\rightarrow$ Last_End, start $\rightarrow$ First_Start, else classes/hours) and sort (most/least, earliest/latest)
7: **end if**
8: **if** $o =$ Count and S returns rooms/professors/courses **then return** P(Count over S)
9: **end if**
10: **if** $o = \perp$ and $k = \perp$ and $w \neq$ Before **then return** $P(S)$ ▷ specialised operation unchanged
11: **end if**
12: **return** composed $P(o$ or List$, k, w,$ filters, when, sort$)$

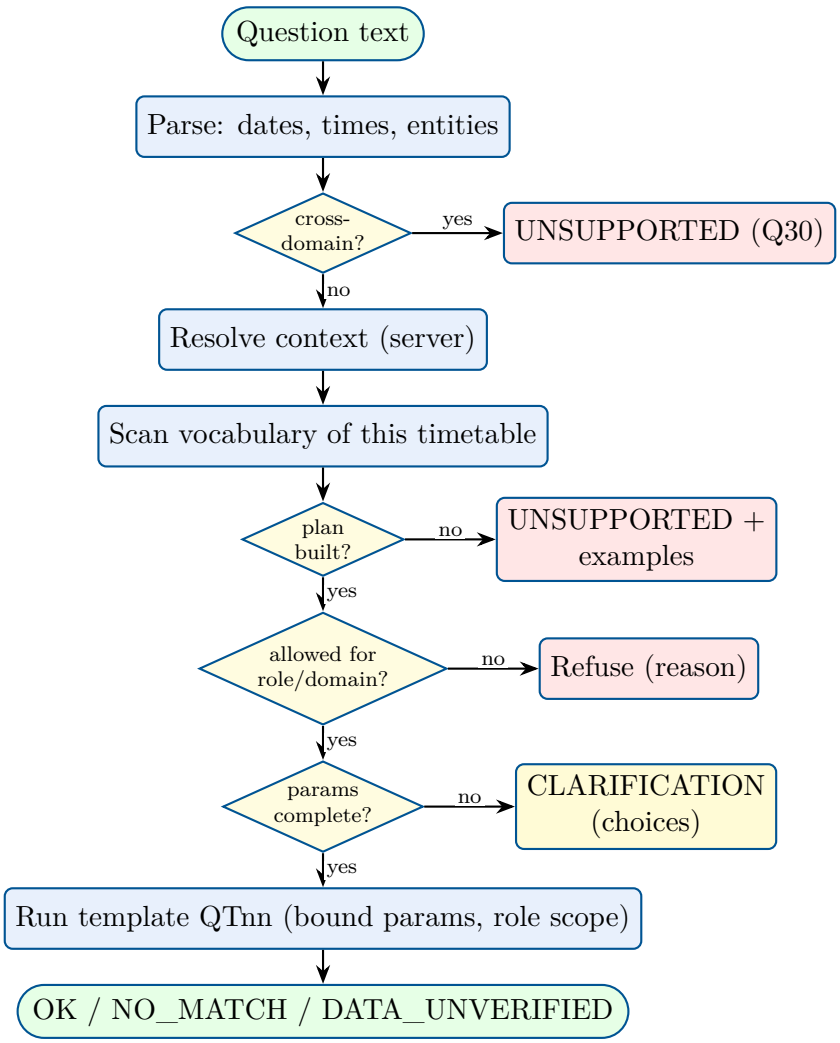


Figure 4.2: Flowchart of the query engine

4.7 Security Design

Passwords are hashed with Argon2id [15]. Access tokens are JWTs [18] valid for 15 minutes; refresh tokens rotate, are stored only as hashes, and reuse of a revoked token revokes the whole family. Roles are read from the database on every request, never from the client. Requests for another user's personal timetable return 404 and are logged. Table 4.1 lists the main threats and controls.

Table 4.1: Threats and controls

Threat	**Control in the design**
Password theft	Argon2id hashing [15]; generic login error; per-IP and per-e-mail rate limit
Stolen refresh token	Rotation on every refresh; reuse of an old token revokes the whole token family
Viewing policy	The official timetable is public, so every signed-in user may view and ask about all of it; roles only decide what "my" means and who may change data (Admin)
Role escalation	Roles read from `USER_ROLE` on every request; role change bumps `token_version`
IDOR on personal data	Owner check in the query itself; non-owners get 404 (no existence leak) and an `ACCESS_DENIED` audit row
SQL / prompt injection	No SQL built from text; query plans use a closed vocabulary of operations and filters; typed, bound parameters; unknown parameters rejected
Malicious upload	Extension allow-list *and* magic-byte signature check; size/page limits; private storage; parsing isolated in the worker; LibreOffice with isolated profile and timeout
Account takeover via reset	Reset codes never returned in API responses or logs in production (bug P0-1 found and fixed in our audit)
Information leakage	Uniform error envelope with request id; no stack traces or SQL in responses; no tokens or free text in logs
Secrets in the app	Only the public API URL is in the bundle; tokens in iOS Keychain / Android Keystore (SecureStore)

Chapter 5: Results and Discussion

5.1 Implementation

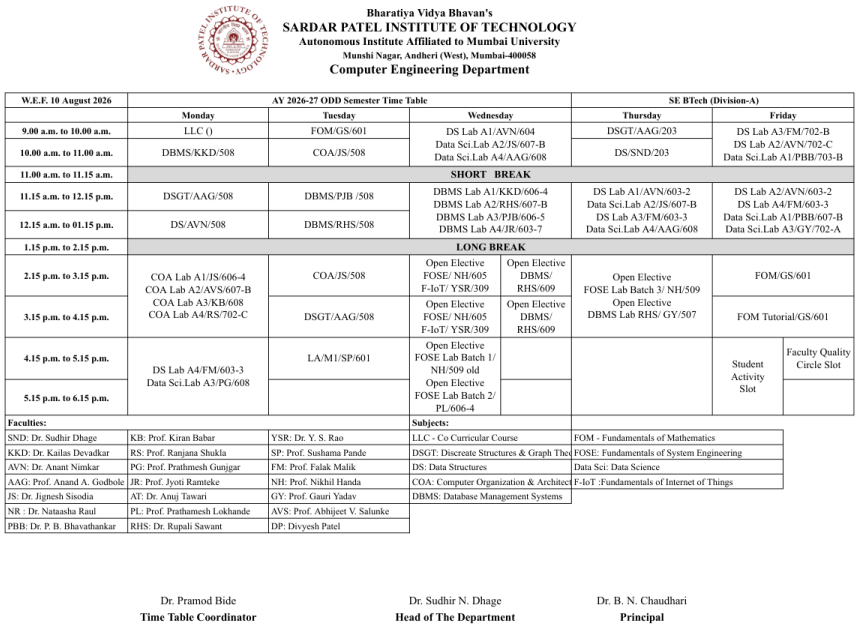
The back end has about 10.5k lines of Python and the app about 5.5k lines of strict TypeScript. The code is in the project's Git repository (`https://github.com/shivammulani-24/Dynamic_Timetable`). Work was divided by module as shown in Table 5.1.

Table 5.1: Contribution of group members

Member	Main contributions
Shivam Mulani (2024300141)	System architecture and database design (schema, migrations, indexes); authentication and role-based access control; natural-language query engine (intent registry Q01–Q30, templates QT01–QT22, clarifications); concurrency control; security audit and fixes; cloud deployment on Render.
Gurshaan Nandrajog (2024300146)	Timetable extraction pipeline (PDF tables, OCR, Excel/Word adapters); tuning on the college's real PDF (lab groups, legends, noon row, combined B.E. page); background worker; conflict detection; extraction and regression tests; performance and load measurements.
Harshvarddhan Waghela (2024300272)	Cross-platform app (React Native/Expo): role dashboards, weekly view, Ask screen, upload, review and admin screens; laptop/web support and Expo SDK upgrade; mobile and end-to-end tests; documentation, report and poster.
All members	Requirements analysis and SRS, design reviews with the guide, testing on phones and laptops, demo preparation.

5.2 Screenshots

Figures 5.1–5.7 show the real input and the running app on a phone and a laptop.



Bharatiya Vidya Bhavan's
SARDAR PATEL INSTITUTE OF TECHNOLOGY
Autonomous Institute Affiliated to Mumbai University
Munshi Nagar, Andheri (West), Mumbai-400058
Computer Engineering Department

W.E.F. 10 August 2026	AY 2026-27 ODD Semester Time Table			SE BTech (Division-A)	
	Monday	Tuesday	Wednesday	Thursday	Friday
9.00 a.m. to 10.00 a.m.	LLC ()	FOM/GS/601	DS Lab A1/AVN/604 Data Sci.Lab A2/JS/607-B Data Sci.Lab A4/AAG/608	DSGT/AAG/203	DS Lab A3/FM/702-B DS Lab A2/AVN/702-C Data Sci.Lab A1/PBB/703-B
10.00 a.m. to 11.00 a.m.	DBMS/KKD/508	COA/JS/508		DS/SND/203	
11.00 a.m. to 11.15 a.m.	SHORT BREAK				
11.15 a.m. to 12.15 p.m.	DSGT/AAG/508	DBMS/PJB /508	DBMS Lab A1/KKD/606-4 DBMS Lab A2/RHS/607-B DBMS Lab A3/PJB/606-5 DBMS Lab A4/JR/603-7	DS Lab A1/AVN/603-2 Data Sci.Lab A2/JS/607-B DS Lab A3/FM/603-3 Data Sci.Lab A4/AAG/608	DS Lab A2/AVN/603-2 DS Lab A4/FM/603-3 Data Sci.Lab A1/PBB/607-B Data Sci.Lab A3/GY/702-A
12.15 a.m. to 01.15 p.m.	DS/AVN/508	DBMS/RHS/508			
1.15 p.m. to 2.15 p.m.	LONG BREAK				
2.15 p.m. to 3.15 p.m.	COA Lab A1/JS/606-4 COA Lab A2/AVS/607-B COA Lab A3/KB/608 COA Lab A4/RS/702-C	COA/JS/508	Open Elective FOSE/ NH/605 F-IoT/ YSR/309 Open Elective DBMS/ RHS/609	Open Elective FOSE Lab Batch 3/ NH/509 Open Elective DBMS Lab RHS/ GY/507	FOM/GS/601
3.15 p.m. to 4.15 p.m.		DSGT/AAG/508	Open Elective FOSE/ NH/605 F-IoT/ YSR/309 Open Elective DBMS/ RHS/609		FOM Tutorial/GS/601
4.15 p.m. to 5.15 p.m.	DS Lab A4/FM/603-3 Data Sci.Lab A3/PG/608	LA/M1/SP/601	Open Elective FOSE Lab Batch 1/ NH/509 old Open Elective FOSE Lab Batch 2/ PL/606-4		Student Activity Slot / Faculty Quality Circle Slot
5.15 p.m. to 6.15 p.m.					

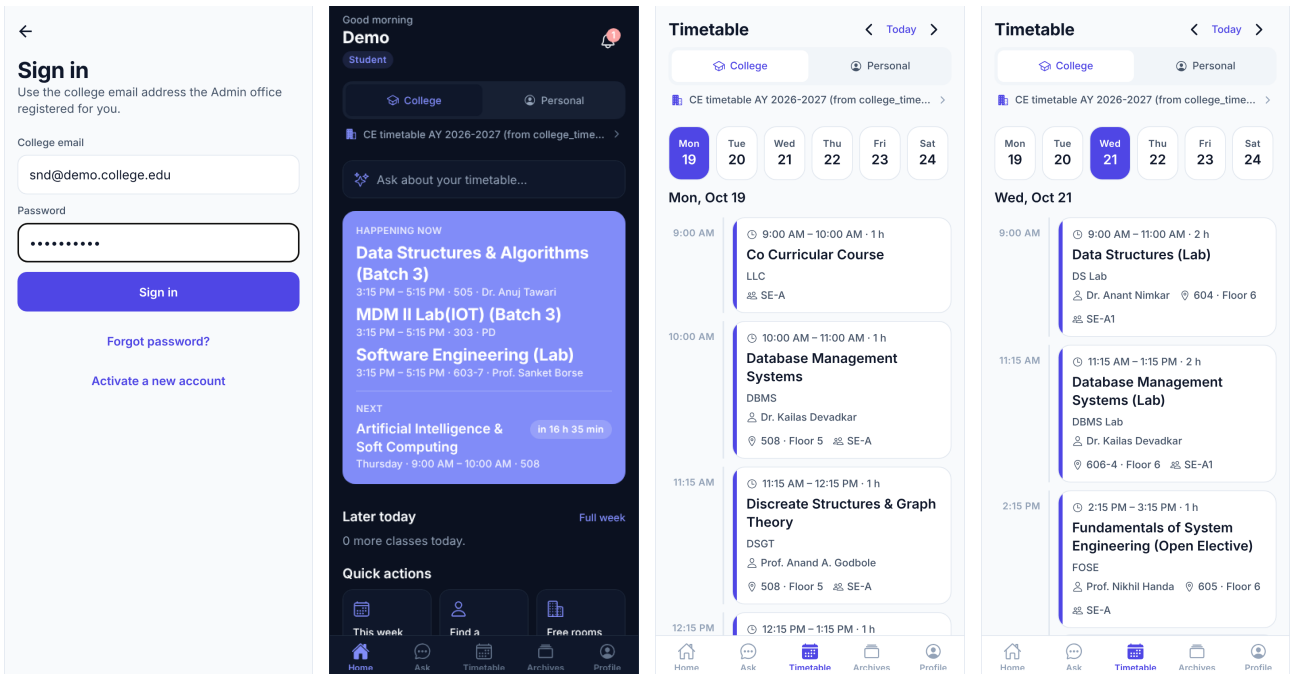
Faculties:			Subjects:	
SND: Dr. Sudhir Dhage	KB: Prof. Kiran Babar	YSR: Dr. Y. S. Rao	LLC - Co Curricular Course	FOM - Fundamentals of Mathematics
KKD: Dr. Kailas Devadkar	RS: Prof. Ranjana Shukla	SP: Prof. Sushama Pande	DSGT: Discreate Structures & Graph Theo	FOSE: Fundamentals of System Engineering
AVN: Dr. Anant Nimkar	PG: Prof. Prathmesh Gunjgar	FM: Prof. Falak Malik	DS: Data Structures	Data Sci: Data Science
AAG: Prof. Anand A. Godbole	JR: Prof. Jyoti Ramteke	NH: Prof. Nikhil Handa	COA: Computer Organization & Architect	F-IoT :Fundamentals of Internet of Things
JS: Dr. Jignesh Sisodia	AT: Dr. Anuj Tawari	GY: Prof. Gauri Yadav	DBMS: Database Management Systems	
NR : Dr. Nataasha Raul	PL: Prof. Prathamesh Lokhande	AVS: Prof. Abhijeet V. Salunke		
PBB: Dr. P. B. Bhavathankar	RHS: Dr. Rupali Sawant	DP: Divyesh Patel		

Dr. Pramod Bide
Time Table Coordinator

Dr. Sudhir N. Dhage
Head of The Department

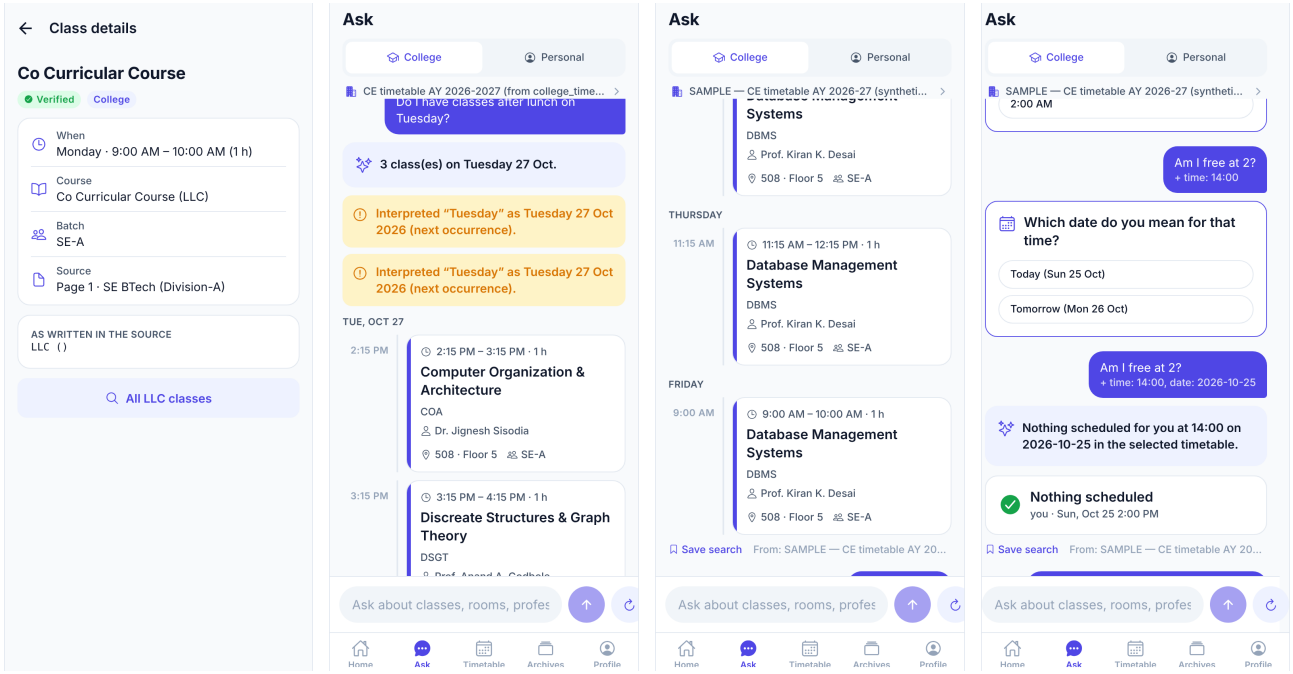
Dr. B. N. Chaudhari
Principal

Figure 5.1: Input: page 1 of the college's real timetable PDF (S.E. Division A), with stacked lab cells and legend



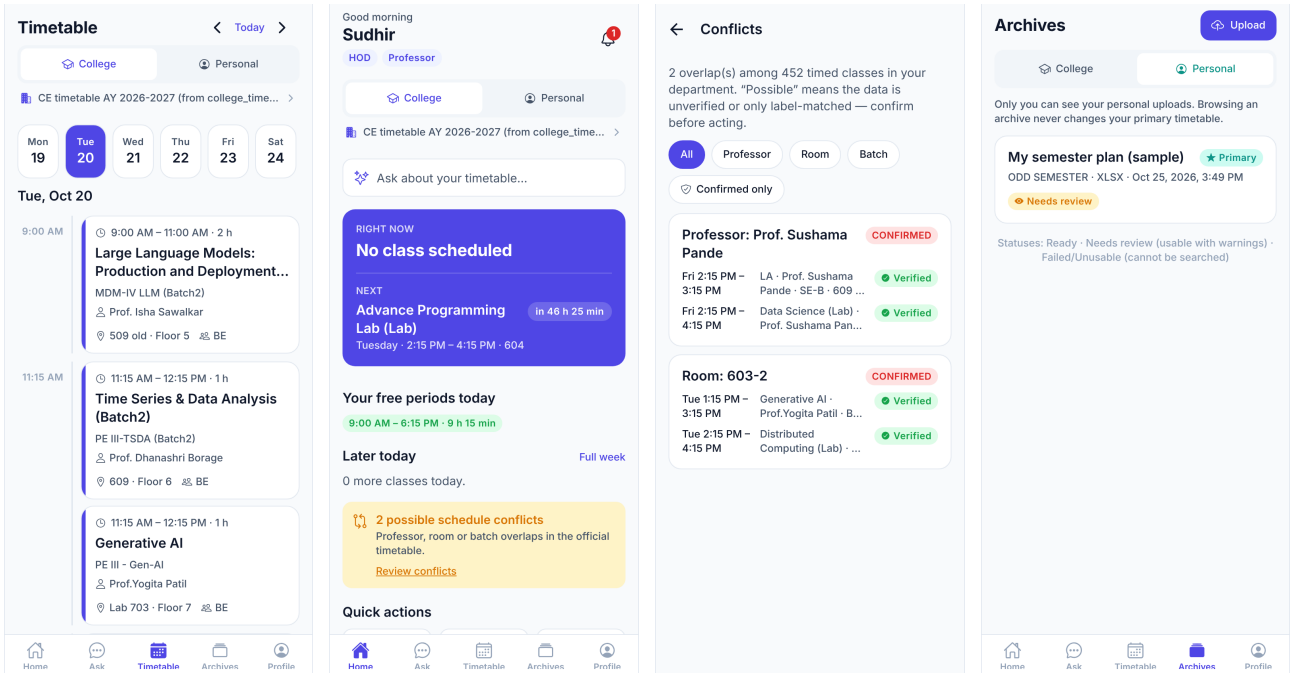
(a) Sign in **(b)** Student home (dark mode): now / next **(c)** Week view, Monday (SE-A1) **(d)** Only lab group A1's labs

Figure 5.2: Student screens



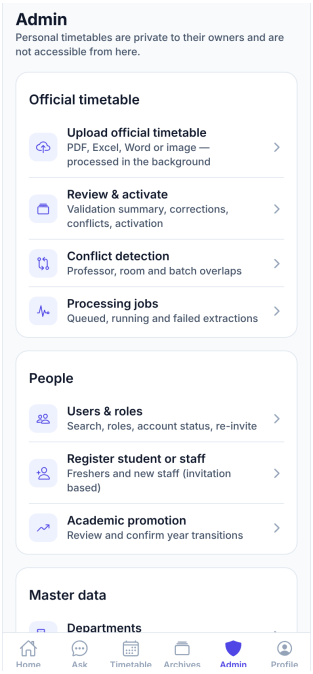
(a) Class details with source page and raw text (LLC: no teacher/room) **(b)** "Classes after lunch on Tuesday?" uses SE lunch **(c)** "Am I free at 2?" asks a.m. or p.m. **(d)** After choosing 2 PM and the date: answer

Figure 5.3: Natural-language search (Ask tab)

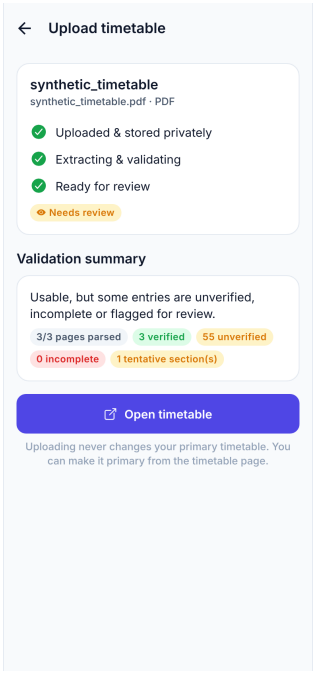


(a) B.E. combined electives **(b)** HOD home with conflict alert **(c)** The 2 genuine clashes **(d)** Personal archives (private)

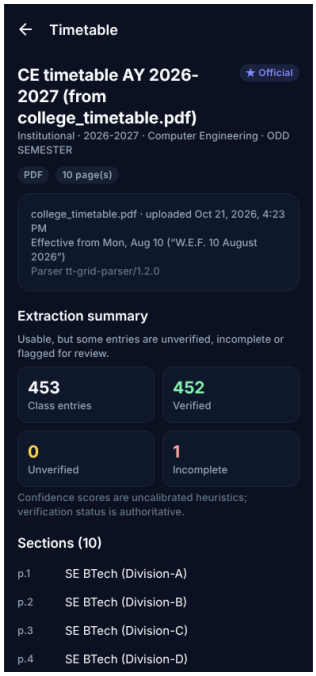
Figure 5.4: Combined classes, HOD view, conflicts and personal domain



(a) Admin hub

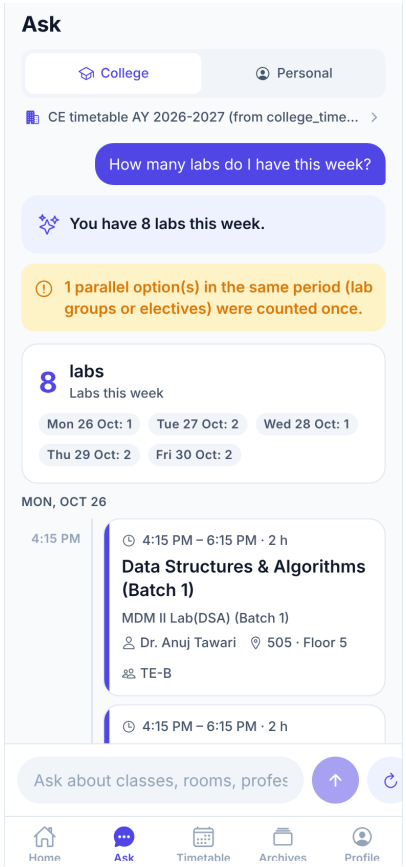


(b) Upload processed by worker

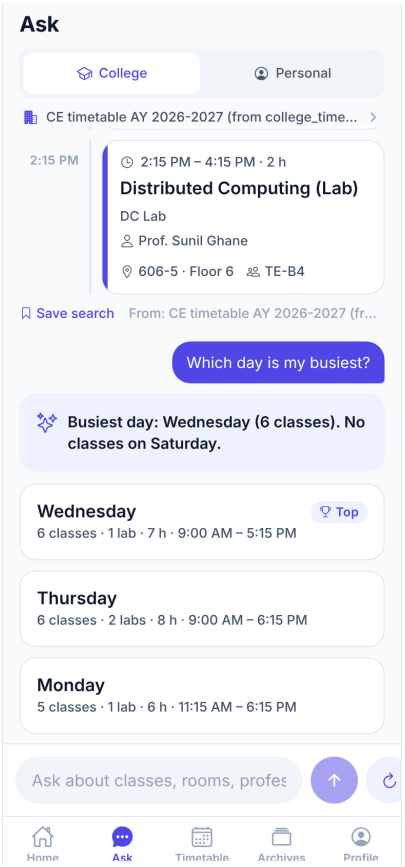


(c) Real PDF: 453 entries, 452 verified, 10 sections

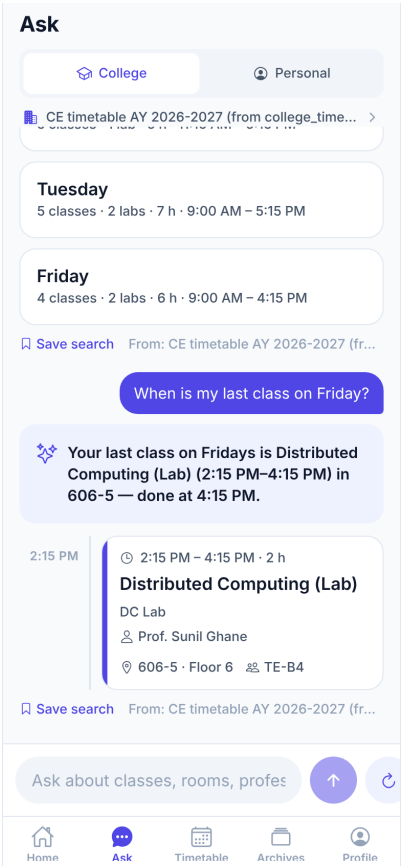
Figure 5.5: Administrator workflow



(a) "How many labs do I have this week?": count with per-day breakdown



(b) "Which day is my busiest?": day-wise summary



(c) "When is my last class on Friday?"

Figure 5.6: Composed query plans answered from the real timetable (TE-B4 student)

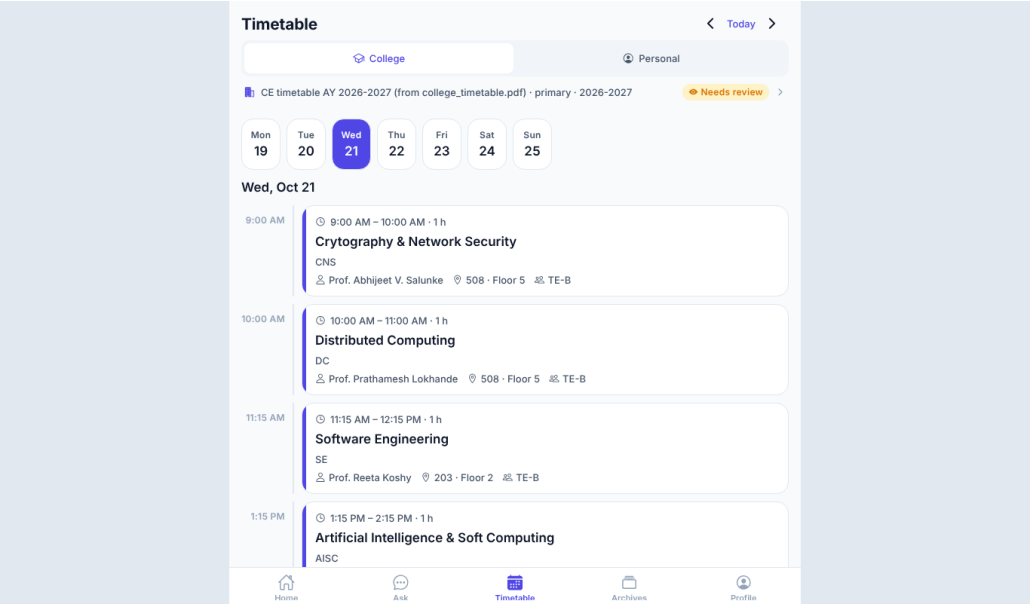


Figure 5.7: The same app in a laptop browser

5.3 Extraction Results

The real 10-page PDF produced 570 rows: 453 classes, of which **452 were verified (99.8%)** and one was INCOMPLETE (an M.Tech. row with no printed time, flagged instead of guessed), plus breaks and activities. All 10 pages were parsed and 46 batches, 38 courses, 52 faculty and 42 rooms were recognised. Table 5.2 gives the extraction time.

Table 5.2: Extraction performance on the real 10-page PDF (median of 5 runs)

Metric	Value
Pages / rows produced	10 / 570 (453 classes)
Extraction time, whole pipeline	1.65 s (runs: 1.72, 1.65, 1.64, 1.69, 1.64 s)
Per page	≈0.17 s
Peak memory	82 MB

5.4 Clash Detection Results

A naive pairwise check reported 542 overlaps, almost all intended (parallel lab batches, electives, combined sessions). The intended-overlap rules reduced this to 25 and then to exactly **2 genuine clashes** (Figure 5.8), both confirmed by hand in the PDF: (1) *Friday 2:15–3:15*, Prof. Sushama Pande is assigned to SE-B in 609 and SE-C's Data Science lab in 606-4; (2) *Tuesday 2:15–3:15*, room 603-2 is used by a B.E. elective and a T.E. lab.

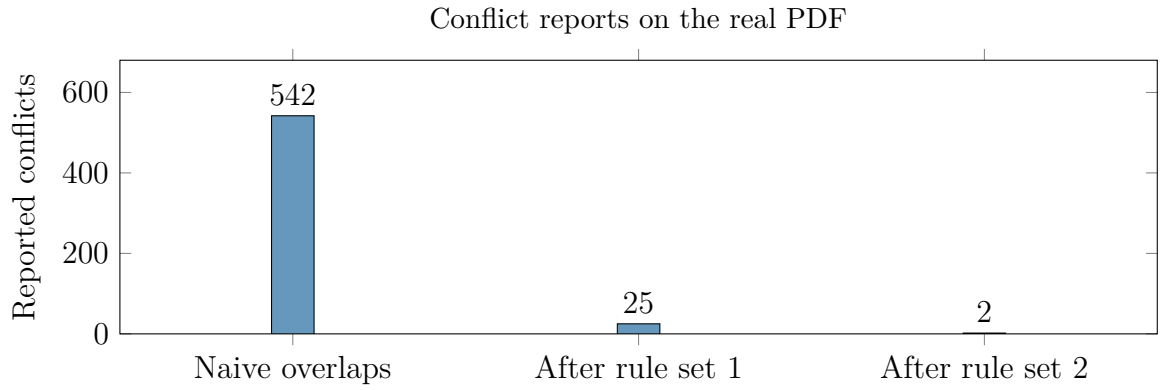


Figure 5.8: Effect of intended-overlap rules: from 542 raw overlaps to the 2 genuine clashes

5.5 Query Results and Performance

Questions are answered in 37–47 ms (median) for a single user (Table 5.3). Under load, throughput peaks at about 60 requests per second with 8 parallel clients, and even at 32 clients every answer was correct (Table 5.4, Figure 5.9). The API worker uses about 115 MB of memory and the whole database is 12 MB.

Table 5.3: Single-user search latency per question type (50 requests each)

Intent	Question	Results	p50 (ms)	p95 (ms)
Q10 Next class	What is my next class?	1	42	52
Q02 Day timetable	What are my classes on Monday?	5	44	52
Q06 Course search	Find all DBMS classes	7	45	51
Q11 After lunch	Do I have classes after lunch on Tuesday?	3	47	54
Q15 Professor schedule	What is my schedule on Monday?	3	37	46
Q22 Free rooms	Which rooms are free at 10 am today?	20	43	51
Q20 Room schedule	Show the schedule of room 508 on Monday	8	41	49

Table 5.4: Load test with increasing concurrent clients (4 API workers)

Clients	Requests	Wrong	req/s	p50 (ms)	p95 (ms)	CPU (%)
1	30	0	18.6	49	57	31
2	60	0	25.4	75	90	32
4	120	0	46.5	64	90	48
8	240	0	60.4	90	141	63
16	480	0	54.1	183	354	62
32	960	0	57.4	410	635	76

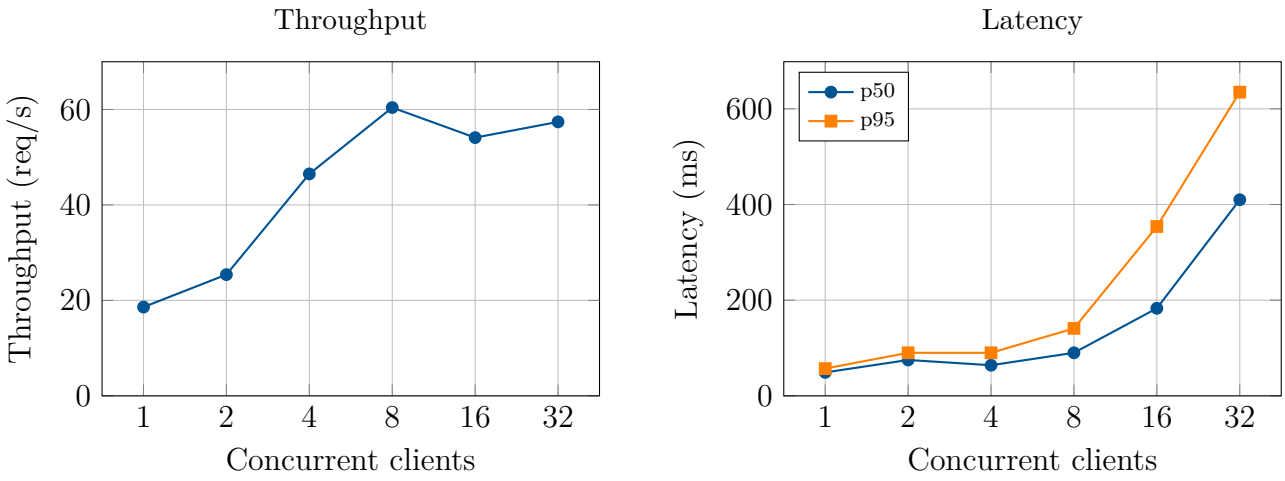


Figure 5.9: Throughput and latency under increasing concurrent load (measured)

5.6 Testing

179 back-end tests run against a real PostgreSQL database, the app has 20 Jest tests and compiles under strict TypeScript, and 45 end-to-end checks run against the live API; all pass. Table 5.5 lists the main test cases.

Table 5.5: Test case specification

TC	Req.	Scenario (input)	Expected result	Actual	P/F	Evidence
TC-01	FR-01	Wrong password vs unknown user (Two bad logins)	Same generic error	As expected	P	test_auth
TC-02	FR-01	Refresh-token reuse (Reuse rotated token)	Whole family revoked, 401	As expected	P	test_auth
TC-03	FR-02	Role escalation (Student sends `roles:[ADMIN]`)	Ignored; admin APIs 403	As expected	P	test_auth

TC	Req.	Scenario (input)	Expected result	Actual	P/F	Evidence
TC-04	FR-03	Wrong file type (Text file named `.pdf`)	Rejected, nothing stored	As expected	P	test_ extraction
TC-05	FR-03	Upload does not change primary (Upload new file)	Primary unchanged	As expected	P	test_uploads
TC-06	FR-03	Retry same upload (Same client_request_id twice)	One archive record	As expected	P	test_uploads
TC-07	FR-04	Real PDF extraction (10-page college PDF)	≥95% verified	452/453 (99.8%)	P	test_college_ seed
TC-08	FR-04	Stacked lab cell (Cell with A1–A4)	4 entries, own room/teacher	As expected	P	test_ reference_pdf
TC-09	FR-04	Noon row typo (“12.15 a.m. to 01.15 p.m.”)	12:15–13:15 with visible note	As expected	P	test_ reference_pdf
TC-10	FR-04	Scanned PDF (Image-only PDF)	OCR, entries UNVERIFIED	As expected	P	test_ extraction
TC-11	FR-05	No-table PDF (Plain text PDF)	UNUSABLE, cannot be primary	As expected	P	test_ extraction
TC-12	FR-06	Correction audited (Admin edits room)	Original kept, audit row	As expected	P	test_uploads
TC-13	FR-07	Concurrent activation (2 threads activate)	Exactly one default	As expected	P	test_ academics
TC-14	FR-09	Dashboard = query (Home vs “next class?”)	Identical entry	As expected	P	test_query_ engine
TC-15	FR-10	Lab group isolation (SE-A1, Wednesday)	Only A1 labs	As expected	P	test_college_ seed
TC-16	FR-11	Lunch per year (SE vs TE “after lunch”)	14:15 vs 13:15 from PDF	As expected	P	test_college_ seed
TC-17	FR-11	Ambiguous time (“Am I free at 2?”)	Clarification a.m./p.m.	As expected	P	test_query_ engine, e2e
TC-18	FR-11	Injection text (“’; DROP TABLE . . . ”)	Treated as text; no SQL run	As expected	P	test_query_ engine
TC-19	FR-11	Cross-domain compare (Q30 question)	UNSUPPORTED, no query executed	As expected	P	test_query_ engine
TC-20	FR-12	Free rooms on a floor (“free rooms on floor 5 at 10”)	Rooms without overlapping class	As expected	P	test_query_ engine
TC-21	FR-13	Clashes in real PDF (Real timetable)	Exactly 2 genuine clashes	2 (SP Fri; 603-2 Tue)	P	test_college_ seed
TC-22	FR-14	IDOR personal (Other user’s timetable id)	404 + audit	As expected	P	test_uploads
TC-23	FR-15	Promotion (Preview + confirm)	History kept, new batch	As expected	P	test_ academics

TC	Req.	Scenario (input)	Expected result	Actual	P/F	Evidence
TC-24	FR-15	Concurrent promotion (2 threads)	One open placement	As expected	P	test_ academics
TC-25	FR-17	Student without batch (“next class?”)	Setup-required message	As expected	P	test_college_ seed
TC-26	NFR-04	Reset code in prod (Production, no SMTP)	Code never in response/log	As expected	P	test_deploy
TC-27	NFR-03	Connectivity probe (`HEAD /`)	200 (not 405)	As expected	P	test_deploy
TC-28	NFR-01	Search latency (50 req/intent)	p50 $<$ 300 ms	37–47 ms	P	Table 5.3
TC-29	NFR-03	Load (32 clients, 960 req)	0 wrong answers	0	P	Table 5.4
TC-30	UI	Full student/HOD flow (Browser, real data)	All steps pass	24/24	P	college.e2e.mjs
TC-31	FR-11	Count with filters (“How many labs do I have this week?”)	Count = distinct lab periods; only labs	As expected	P	test_college_ seed
TC-32	FR-11	First/last (“When is my last class on Friday?”)	Same as max end of the Friday listing	As expected	P	test_college_ seed
TC-33	FR-11	Day-wise ordering (“Which day is my busiest / finish earliest?”)	Days sorted by classes / finish time	As expected	P	test_college_ seed
TC-34	FR-02	“My labs” vs another batch (“Show my labs”, “Show SE-B labs”)	Own lab group for “my”; SE-B shown when asked	As expected	P	test_college_ seed
TC-35	FR-11	Lunch and free periods (“When is lunch break on Monday?”, “When am I free on Thursday?”)	Own section’s lunch (TE 12:15, SE 1:15); gaps between own classes	As expected	P	test_college_ seed
TC-36	FR-11	Who teaches / what comes after / which floor (“Who teaches software engineering?”)	Teacher names; next class after; floor of room	As expected	P	test_college_ seed
TC-37	FR-02	Open viewing (student asks about SE-B, floor 5, clashes)	Shown to everyone; “my” still own batch	As expected	P	test_college_ seed
TC-38	FR-11	Short words not read as names (“Which room is my next class in?”)	Next class, not a batch called “in”	As expected	P	test_college_ seed

5.7 Comparative Analysis

Compared with the existing approaches in Table 2.1, the proposed system is the only one that reads the published timetable and also answers questions with reproducible, injection-safe results. Table 5.6 maps each objective to its evidence.

Table 5.6: Objectives mapped to evidence

Obj.	Evidence	Status
O1	99.8% verified on the real PDF; uncertain values flagged (TC-07, TC-09)	Achieved
O2	PDF, scanned PDF/photo (OCR), XLSX/XLS, CSV, DOCX/DOC through one pipeline (TC-04, TC-10)	Achieved
O3	Five roles, server-side policy, role tests (TC-03), dashboards (Fig. 5.2, 5.4)	Achieved
O4	Specialised operations plus composed query plans (counts, first/last, day-wise, lab/lecture), with clarification (TC-16–TC-19, TC-31–TC-34)	Achieved
O5	2 genuine clashes, 0 false reports (TC-21)	Achieved
O6	Separate tables; IDOR and isolation tests (TC-22); archives	Achieved
O7	p50 37–47 ms; 0 wrong answers at 32 clients (TC-28, TC-29)	Achieved (dev hardware)
O8	Android/iPhone via Expo Go, laptop browser, Render demo	Achieved; store builds pending
O9	179 + 20 + 45 e2e steps passing	Achieved

5.8 Improvements Incorporated after Review

The following improvements were made after the project reviews and are part of the version demonstrated at the ESE (Table 5.7). Each one is covered by automated tests.

Table 5.7: Improvements incorporated before the ESE

Area	Before	After (current version)
Question coverage	A fixed list of about 30 question types; counts, "busiest day" or "last lab on Friday" were refused	Compositional query planner: 17 operations combined with filters, dates, 6 time windows and sorting (Section 5, TC-31–TC-36)
Teacher names	Only faculty codes such as AVN were understood	Code, first name, surname or "Nimkar sir" all resolve to the same person; spelling variants sharing one code are merged
Subject names	Exact subject names only	Partial words and one-letter typos (e.g. the PDF's own "Discreate") are matched
Viewing other timetables	Students saw only their own batch	Anyone can view any teacher's, batch's or room's official timetable through a picker; personal timetables stay private (TC-37)
Lunch and free time	Not answerable	Each section's own lunch and the free gaps between one's classes (TC-35)
Short words in questions	"in", "at" or "is" were sometimes read as a batch or a name	Stop-word lists prevent these misreadings (TC-38)
Explainability	Only the answer was shown	Every answer carries the query plan that produced it

5.9 Scenario-Based Challenges

Table 5.8 lists realistic situations the system can face in a college and how it handles each one; every row is backed by a test case or a measured result.

Table 5.8: Scenario-based challenges and how the system responds

Scenario	System behaviour	Evidence
The department releases a revised timetable mid-semester	The admin uploads the new PDF; the old version stays in the archive and the current one keeps serving users until the new one is reviewed and activated	TC-05, TC-06
A room changes for one class after publishing	The admin corrects the entry; the original value, the new value, the reason and the admin are stored in an audit record	TC-12

Scenario	System behaviour	Evidence
The PDF prints the noon row as "12.15 a.m. to 01.15 p.m."	Backtracking proves 12:15–13:15 from the neighbouring rows and shows a visible note	TC-09, Alg. 1
A row has no printed time	The entry is marked INCOMPLETE and is not used in answers, instead of being guessed	Sec. 10.3
Only a phone photo or scanned copy is available	OCR reads it and its entries are marked UNVERIFIED for review	TC-10
A wrong file is uploaded (text renamed to .pdf, or a PDF without a table)	Rejected or marked UNUSABLE; it can never become the official timetable	TC-04, TC-11
The same teacher is spelled two ways in the PDF	Entries sharing a faculty code are merged into one person	TC-36
One teacher is scheduled in two rooms at once	Reported as a clash to the HOD; the 2 genuine clashes in the real PDF were found	TC-21
A student types "Am I free at 2?"	The system asks whether 2 a.m. or 2 p.m. is meant instead of guessing	TC-17
A user types SQL such as `'; DROP TABLE` into the question box	Treated as plain text; no SQL is built from user text	TC-18
A student tries to open another user's personal timetable	Returns 404 and the attempt is logged	TC-22
A student sends `roles:[ADMIN]` in a request	Ignored; admin endpoints return 403	TC-03
A stolen refresh token is reused	The whole token family is revoked	TC-02
Two admins activate different timetables at the same moment	Exactly one becomes the default; no corrupted state	TC-13
Year-end promotion of all students	Preview, then confirm; history is kept and each student has one current placement	TC-23, TC-24
A new student has not been assigned a batch yet	A clear "setup required" message instead of wrong classes	TC-25

Scenario	System behaviour	Evidence
Many students open the app at 8:55 a.m.	32 parallel clients: 57 requests/s, 0 wrong answers	TC-29

5.10 Discussion

- **Real data differs from specifications.** Most extraction work went into conventions found only in the real PDF: the mislabelled noon row, codes missing from legends, LLC slots and the combined B.E. page. Testing on the real document from the start was decisive.
- **Honest uncertainty builds trust.** Marking the one entry without a time as INCOMPLETE, instead of guessing, kept every automatic answer correct.
- **The source document itself had errors.** The two clashes are real errors in the published timetable, so the system already adds value for the timetable committee.
- **Rules work here if they compose.** A fixed list of question types was limiting; composing rules into a query plan removed most of that limit while keeping exact answers in about 40 ms, at no API cost and with no injection risk.

Chapter 6: Conclusion and Further Work

6.1 Conclusion

The college timetable is consulted by everyone every day, yet it exists only as a dense PDF with no search, no personal view, no version control and no clash checking. *Dynamic Timetable* reads the timetable as it is actually published (PDF, scanned image, Excel or Word), extracts every class with its source page and verification status, detects clashes, and serves students, professors, the HOD, the Principal and the Admin through one app on Android, iPhone and laptop.

On the department's real 10-page timetable the system verified 452 of 453 classes (99.8%), reduced 542 raw overlaps to the 2 genuine clashes, and answered plain-English questions in about 40 ms with no wrong answers under 32 concurrent clients. A constrained backtracking algorithm resolves inconsistent time labels with certainty, a group-sort-sweep detector separates intended overlaps from real clashes, and a deterministic compositional planner answers a wide range of questions while never generating SQL from user text and asking for clarification instead of guessing. All nine objectives were met, and the whole system runs at zero cost on open-source software and a free cloud tier.

6.2 Limitations

- Extraction accuracy was assessed by verification status and manual checks, not by precision and recall on independently hand-labelled pages.
- The load test ran on development hardware, not on the production host.
- Some short forms (e.g. "AI" for an elective) are not recognised yet, and a bare time such as "at 2" always asks a.m. or p.m.
- No Play Store / TestFlight build or push notifications yet.

6.3 Further Work

1. Measure per-field precision and recall on hand-labelled timetable pages.
2. College single sign-on (Google Workspace) instead of separate passwords.
3. Store builds with push notifications for timetable changes and "class in 10 minutes" reminders.
4. "What changed?" comparison between two official versions, sent only to affected users.

5. An optional LLM that *suggests* query plans for unusual phrasings, still checked by the same validator (never free SQL).
6. Bulk CSV import, room-utilisation analytics for the HOD and Principal, and exam timetables.
7. Capacity testing on the production host.

Chapter 7: Research Paper/ Poster

7.1 Research Paper

A research paper based on this project was prepared in the IEEE conference format:

> S. Mulani, G. Nandrajog, H. Waghela and A. Nimkar, "Dynamic Timetable: Extracting Published College Timetables and Answering Natural-Language Questions with a Deterministic Compositional Query Planner."

It is reproduced on the following pages. Its plagiarism report is included in Section 8.

Dynamic Timetable: Extracting Published College Timetables and Answering Natural-Language Questions with a Deterministic Compositional Query Planner

Shivam Mulani
Dept. of Computer Engineering
Sardar Patel Institute of Technology
Mumbai, India
UID 2024300141

Gurshaan Nandrajog
Dept. of Computer Engineering
Sardar Patel Institute of Technology
Mumbai, India
UID 2024300146

Harshvarddhan Waghela
Dept. of Computer Engineering
Sardar Patel Institute of Technology
Mumbai, India
UID 2024300272

Prof. Anant Nimkar
Dept. of Computer Engineering
Sardar Patel Institute of Technology
Mumbai, India
(Project Guide)

***Abstract*—College timetables are usually published as dense multi-page PDF grids that students and faculty must read by hand. Such documents are hard for software too: lab cells stack several batches, subject and faculty codes are expanded only in page-local legends, time labels are inconsistent, and names are spelled differently across pages. This paper presents *Dynamic Timetable*, a role-based system that extracts the timetable as it is actually published (PDF, scanned image, spreadsheet or document), detects scheduling clashes, and answers questions in plain English on Android, iOS and the web. We contribute (i) a constrained backtracking algorithm that resolves inconsistent time labels and accepts a reading only when it is identical in every valid assignment, (ii) a group-sort-sweep conflict detector that separates intended overlaps from genuine clashes in $O(n \log n + m)$, and (iii) a deterministic *compositional query planner* that maps a question to a typed plan of operation, filters, date and time window and compiles it to parameterised SQL, so no SQL is ever generated from user text. On a real 10-page Computer Engineering timetable the system verified 452 of 453 classes (99.8%), reduced 542 raw overlaps to the 2 genuine clashes, answered questions with a median latency of 37–47 ms, and returned zero wrong answers under 32 concurrent clients. The full stack is open source and runs on a free cloud tier.**

***Index Terms*—timetable extraction, table recognition, natural language interface to databases, query planning, interval scheduling, conflict detection, role-based access control, mobile applications**

I. Introduction

Every semester an engineering department publishes its timetable, typically as a PDF with one page per class division. Students and faculty consult it several times a day to answer simple questions: *Where is my next class? Is room 508 free at 2 pm? When is Prof. Nimkar free on Monday?* Answering these by hand means locating the right page, row, column and lab batch, and then looking up abbreviations in a legend at the bottom of the page. Clashes, such as one teacher in two rooms at once, are usually discovered only when they happen, and old and revised versions of the PDF circulate together.

Two families of software address parts of this problem. Timetable *generators* such as FET, UniTime and aSc [1]–[3] solve the constraint problem of building a timetable, but they do not read a timetable that has already been published by other means. Generic *table extractors* such as Tabula and Camelot [4], [5] recover cell text but know nothing about lab batches, legends or time rows. On the question-answering side, large language model (LLM) text-to-SQL systems understand free text but can produce incorrect queries and are vulnerable to prompt and SQL injection [6], [7], which is unacceptable when an answer decides where a student walks next.

We present Dynamic Timetable, which starts from the published document and ends with exact answers. The contributions of this paper are:

1) A timetable-aware extraction pipeline with page-local legend expansion, stacked lab-cell parsing, per-entry provenance and a constrained backtracking resolver for inconsistent time labels.
2) A conflict detector that distinguishes intended overlaps (parallel lab batches, elective options, combined sessions) from genuine teacher, room and batch clashes.
3) A deterministic compositional query planner over a closed vocabulary built from the timetable itself, with clarification instead of guessing and an explainable plan attached to every answer.
4) An evaluation on a real 10-page college timetable covering extraction quality, clash detection, latency, concurrency and test coverage.

II. Related Work

A. Automated timetabling

University timetabling has been studied extensively as a constraint satisfaction and optimisation problem [8]–[10]. These methods *construct* timetables; our work is complementary and operates on the timetable after publication, when its consumers need to query it and administrators need to check it.

B. Table recognition and OCR

Table recognition from documents is a long-standing problem [11], [12], with benchmark competitions such as ICDAR 2013 [13]. Tesseract [14] is a widely used open-source OCR engine. General-purpose extractors recover grids well but do not interpret domain semantics; in a timetable, a single cell may encode four lab batches, each with its own teacher, subject and room, and abbreviations are defined only in the legend of the same page.

C. Natural language interfaces to databases

Natural language interfaces to databases (NLIDB) have a long history [15]. Neural text-to-SQL models such as Seq2SQL [16] and benchmarks such as Spider [17] have greatly widened coverage, and LLMs extend it further. However, LLM-integrated applications are exposed to indirect prompt injection [6] and prompt-to-SQL injection [7], and their outputs are not reproducible. For a narrow, high-stakes domain we instead adopt a deterministic planner over a closed vocabulary that compiles only to pre-written parameterised templates.

D. Temporal reasoning and access control

Clash detection rests on interval relations [18]; we use the end-exclusive overlap predicate $a.s < b.e \wedge b.s < a.e$ with sorting and sweeping [19]. Access is governed by role-based access control (RBAC) [20], with credentials hashed by Argon2id [21] and sessions carried in JSON Web Tokens [22], following OWASP guidance [23].

III. System Architecture

Fig. 1 shows the architecture. A single React Native code base built with Expo [24], [25] serves Android, iOS and the web. It communicates over HTTPS with a REST API [26] implemented in FastAPI [27]. All state lives in PostgreSQL 16 [28]: 31 tables in third normal form [29], [30], created by three Alembic migrations. Uploads are processed asynchronously: the upload transaction inserts a job row, and a worker claims it with `SELECT ... FOR UPDATE SKIP LOCKED`, a producer–consumer pattern that needs no separate message broker. The system deploys from a single blueprint file and Docker image to a free platform-as-a-service tier [31].

Business rules are enforced in the schema where possible: `CHECK` constraints on day of week, `end > start`, confidence in $[0, 1]$ and allowed verification states; partial unique indexes for one current academic placement per student and idempotent uploads; and composite foreign keys `(timetable_id, owner_user_id)` so that a user can reference only their own personal timetable. Each extracted entry retains its raw text, source page and confidence for review.

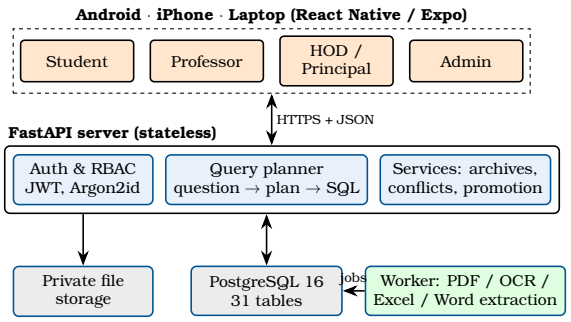


Fig. 1. System architecture: one cross-platform client, a FastAPI back end with a background worker, and PostgreSQL as both database and job queue.

IV. Extraction Pipeline

Fig. 2 summarises the pipeline. PyMuPDF [32] supplies words with coordinates and the ruled lines of the page; scanned pages are passed through Tesseract. Lines are clustered into rows and columns to recover the grid, the day and time headers are located, and each cell is parsed.

A. Cell parsing and legends

A cell is split into lines, and lines belonging to one lab subgroup (e.g. `A1 COA Lab JS 606-4`) are grouped. Within a group, a batch token matches `[A-D][1-4]`, the room is the last token matching a room pattern, faculty codes are tokens present in the page legend, and the remainder is the subject. Codes are expanded *only* if they appear in the legend of the same page; otherwise the code is kept and an informational flag is raised. Each entry is then validated into one of Verified, Unverified, Incomplete or Rejected.

B. Time-row resolution by constrained backtracking

Printed time labels are unreliable. The real document labels the noon row "12.15 a.m. to 01.15 p.m." and many labels omit a.m./p.m. altogether. Algorithm 1 converts each label into a small set of plausible intervals (between 07:00 and 21:00, at most six hours long) and enumerates assignments in which no row starts before the previous row ends. A reading is marked *certain* only if it is the same in every valid assignment; otherwise the row is flagged for review rather than guessed. The noon row is therefore accepted as 12:15–13:15 because its neighbours end at 12:15 and start at 13:15, not because the rule assumes it.

Because the search is bounded at 64 complete assignments, its cost is $O(64r)$ for r rows, negligible compared with page rendering.

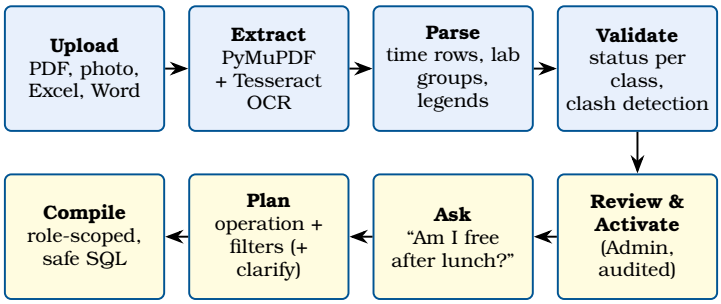


Fig. 2. Extraction pipeline from upload to a verified, published timetable.

Algorithm 1 Resolve a column of time labels

Require: labels $L_1..L_r$ from top to bottom
Ensure: per row: interval and certainty, or a review flag

for $i \leftarrow 1$ **to** r **do**
  $O_i \leftarrow$ plausible readings (s, e) of L_i
  if $O_i = \emptyset$ **then** $O_i \leftarrow$ non-literal readings
  end if
end for
$S \leftarrow \emptyset$
procedure WALK(i, p, A)
  if $|S| > 64$ **then return**
  end if
  if $i > r$ **then** $S \leftarrow S \cup \{A\}$; **return**
  end if
  for all $(s, e) \in O_i$ **with** $s \geq p$ **do**
    WALK($i + 1, e, A \cdot (s, e)$)
  end for
end procedure
WALK($1, 0, \langle\rangle$)
for $i \leftarrow 1$ **to** r **do**
  $R_i \leftarrow \{A_i : A \in S\}$
  if $|R_i| = 1$ and literal **then** row i is CERTAIN
  else if $|R_i| = 1$ **then** keep, flag `TIME_LABEL_INCONSISTENT`
  else leave empty, flag `TIME_AMBIGUOUS_AMPM`
  end if
end for

V. Conflict Detection

A naive pairwise check is both slow, $O(n^2)$, and noisy, because timetables contain many *intended* overlaps. Algorithm 2 groups entries by resource (teacher, room or batch) and day, sorts each group by start time, and sweeps forward, stopping as soon as a later entry starts after the current one ends. A pair is reported only if the intervals overlap and the overlap is not intended: entries from the same source cell, the same combined session printed on several pages, parallel elective options, or a sub-group's own written period. The total cost is $O(n \log n + m)$ for m reported pairs. A clash is CONFIRMED when both entries are verified, mapped to master data and time-certain, and POSSIBLE otherwise.

Algorithm 2 Group, sort and sweep

Require: entries E with day and times

$G \leftarrow$ hash map from (resource, key, day) to lists
for all $x \in E$, resource (k, key) of x **do**
  append x to $G[(k, key, x.day)]$
end for
for all group $g \in G$ **do**
  sort g by start
  for $i \leftarrow 1$ **to** $|g|$ **do**
    for $j \leftarrow i + 1$ **to** $|g|$ **do**
      if $g_j.s \geq g_i.e$ **then break**
      end if
      if not INTENDED(g_i, g_j) **then** report (k, g_i, g_j)
      end if
    end for
  end for
end for

VI. Compositional Query Planner

A. From fixed intents to composed plans

An initial design mapped each question to one of about thirty fixed intents. This limited users to questions the designers had anticipated. The final design instead recognises *independent pieces* of a question and composes them into a typed plan:

- **Operation**: LIST, COUNT, EXISTS, TOTAL_HOURS, FIRST, LAST, NEXT, PER_DAY, SPAN, FREE_SLOTS, BREAKS, AFTER_COURSE, BEFORE_COURSE, TEACHERS, PER_ROOM, ROOM_INFO, CONFLICTS (17 in total).
- **Filters**: the asking user, a batch, a professor, a room, a course; lab or lecture; room type.
- **When**: a date, a weekday, this or next week, or the whole weekly timetable.
- **Time window**: at, after, before, between, after lunch, or now.
- **Metric and sort** for day-wise questions: most or least classes or hours, earliest or latest start or finish.

For example, “Which day do I finish earliest?” becomes {`op: PER_DAY, filters: me, metric: LAST_END, sort: EARLIEST`}, and “How many labs do I have this week?” becomes {`op: COUNT, kind: LAB, when: {week: this}`}. Questions needing dedicated logic (free rooms, a professor’s free time, archives) keep specialised templates; all other plans are compiled by one composed executor from shared building blocks.

B. Vocabulary and name resolution

Entities are recognised by scanning the question against a vocabulary built from the active timetable itself (about 200 labels for the real document): teacher names and codes, rooms, subjects and batches. Teachers can be referred to by code, first name, surname or “*X* sir/ma’am”; entries that share a faculty code are merged, which reconciles spelling variants such as “Sushama” and “Sushma”. Subjects match on partial words and on words within edit distance one, which handles the document’s own typo “Discreate”. A lab batch such as A2 also includes the lectures of its parent division. Resolution proceeds from exact match to unique word match to fuzzy *suggestions*; the system never silently chooses among several candidates.

C. Safety and explainability

Every plan is validated against a closed, immutable registry of operations and parameter values before execution, and is compiled into pre-written parameterised SQL. User text never reaches the SQL string, so injection is not possible by construction. When a question is genuinely ambiguous, for example “at 2” (a.m. or p.m.) or a surname shared by two teachers, the engine returns a clarification request instead of an answer. Each answer carries its plan in a `meta.plan` field that the app can display, making the result auditable.

D. Access policy

The official timetable is public information within the college, so any authenticated user may view any teacher’s, batch’s or room’s schedule and ask questions about it. Personal timetables are private to their owner; requesting another user’s personal timetable returns `404`. Clash reports are visible to the HOD, Principal and Admin, and only the Admin may upload, correct or publish. Access tokens are valid for 15 minutes; refresh tokens rotate, are stored only as hashes, and reuse of a revoked token revokes the whole token family.

VII. Implementation

The back end comprises about 10.5k lines of Python 3.13 with SQLAlchemy 2 and Alembic; the client comprises about 5.5k lines of strict TypeScript using Expo Router and TanStack Query. The client performs a single-flight token refresh: the first `401` triggers one refresh call and other failing requests wait for it. Fig. 3 shows representative screens: the student home and day view, the Ask screen with composed answers, the timetable picker for viewing any teacher, batch or room, and the administration views.

TABLE I
Effect of intended-overlap rules on reported clashes

Rule set	Reported
Naive pairwise overlap	542
+ same source cell, parallel lab batches	25
+ elective options, combined sessions	2
Genuine clashes (manual check)	2

TABLE II
Single-user latency per question type (ms)

Question	p50	p95
What is my next class?	42	52
What are my classes on Monday?	44	52
Find all DBMS classes	45	51
Do I have classes after lunch on Tuesday?	47	54
(Professor) What is my schedule on Monday?	37	46
Which rooms are free at 10 am today?	43	51
Show the schedule of room 508 on Monday	41	49

VIII. Evaluation

A. Setup

All measurements use the department’s real 10-page Computer Engineering timetable (SE, TE, BE and M.Tech. divisions). Performance was measured on a Linux VM with 4 vCPU and 15 GB RAM, with PostgreSQL 16 on the same machine and four API worker processes.

B. Extraction quality

The pipeline produced 570 rows: 453 classes plus breaks and activities. Of the 453 classes, 452 were Verified (99.8%) and one was Incomplete, an M.Tech. row with no printed time, which the system flagged rather than guessed. The whole document was processed in 1.65 s (five runs: 1.64–1.72 s), about 0.17 s per page, with a peak memory of 82 MB.

C. Conflict detection

A naive pairwise check reported 542 overlaps. Adding the same-cell and lab-batch rules reduced this to 25, and adding elective and combined-session rules reduced it to 2. Both remaining clashes were confirmed by hand against the PDF, giving zero false reports on this document (Table I).

D. Latency and concurrency

Table II reports single-user latency over 50 requests per question type: medians of 37–47 ms and 95th percentiles of at most 54 ms, well below the 300 ms objective. Eliminating N+1 queries by eager loading halved the SQL statements per search from about 50 to 25.

For the load test, N concurrent clients, each logged in as a different real student account, sent 30 back-to-back requests cycling through five realistic questions, and every response was checked for correctness. Table III shows that throughput peaks at about 60 requests per second at eight clients; beyond that, requests queue and latency grows, but no request failed and no answer was wrong even at 32 clients. The API worker

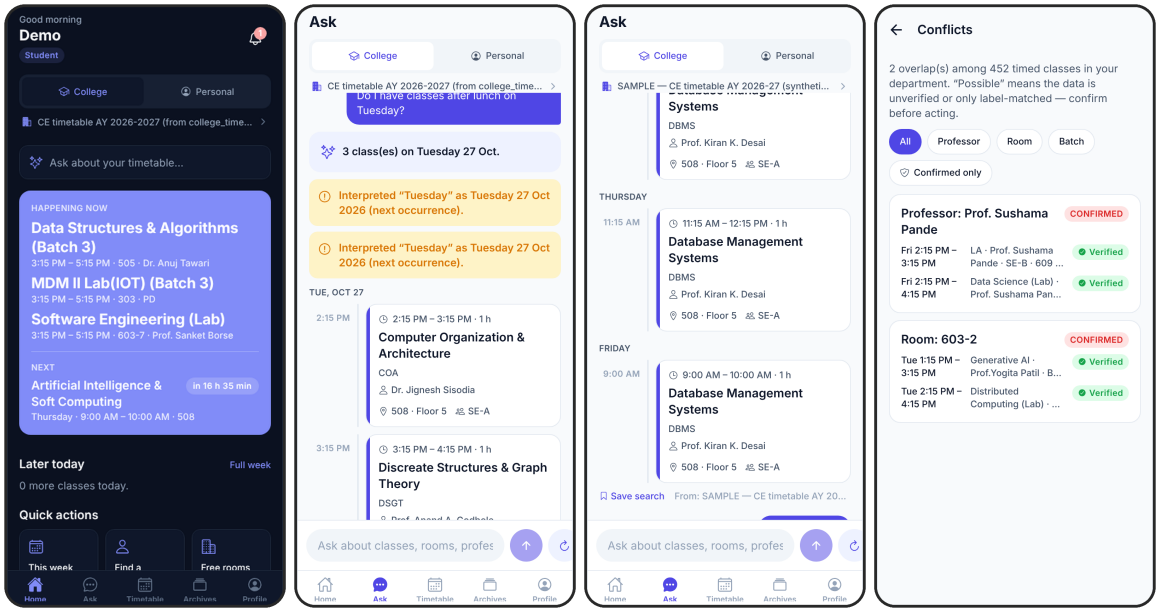


Fig. 3. Representative screens of the mobile and web client.

TABLE III
LOAD TEST WITH INCREASING CONCURRENT CLIENTS

Clients	Wrong	req/s	p50 (ms)	p95 (ms)
1	0	18.6	49	57
2	0	25.4	75	90
4	0	46.5	64	90
8	0	60.4	90	141
16	0	54.1	183	354
32	0	57.4	410	635

TABLE IV
COMPARISON WITH EXISTING APPROACHES

Criterion	Manual	FET/aSc	Tabula	LLM	Ours
Reads published PDF	–	No	Partly	No	Yes
Labs and legends	Human	Yes	No	Depends	Yes
Plain-English questions	No	No	No	Yes	Yes
Reproducible, injection-safe	–	–	–	No	Yes
Clash check after publishing	No	At creation	No	No	Yes
Cost	Free	Free/Paid	Free	Paid API	Free

used 113–116 MB of memory and the database occupied 12 MB.

E. Testing

The back end has 179 automated tests run against a real PostgreSQL instance, covering the query engine (59), the real college timetable (33), extraction (24), authentication and token rotation (15), a reference PDF (14), administration (13), uploads and privacy isolation (13) and deployment (8). The client has 20 Jest tests and compiles under strict TypeScript, and 45 end-to-end checks exercise the running API. All pass.

F. Comparison

Table IV compares the system with existing approaches. It is the only option that reads the published document and also answers questions with reproducible, injection-safe results.

IX. Discussion and Limitations

The deterministic planner trades open-ended coverage for exactness: a phrasing outside the vocabulary yields a clarification or a suggestion rather than a confident wrong answer. In this domain we consider that the right trade-off. Several limitations remain. Extraction accuracy was assessed by verification status and manual checks, not by precision and recall against independently hand-labelled pages. The load test ran on development hardware, not on the free production instance. Some short forms (for example "AI" for an elective) are not yet recognised, and a bare time such as "at 2" always triggers an a.m./p.m. clarification. Store builds and push notifications have not yet been released.

X. Conclusion and Future Work

Dynamic Timetable shows that a published, irregular timetable PDF can be turned into a reliable, queryable service without manual data entry. Constrained backtracking resolves inconsistent time labels with provable certainty, a group-sort-sweep detector isolates the genuine clashes among hundreds of intended overlaps, and a deterministic compositional planner answers a wide range of plain-English questions in tens of

milliseconds while remaining explainable and injection-safe. Future work includes measuring per-field precision and recall on hand-labelled pages, college single sign-on, change alerts sent only to affected users, an optional LLM that *suggests* plans which must still pass the same validator, and capacity testing on production hardware.

Acknowledgment

The authors thank Prof. Anant Nimkar for his guidance and Dr. Sudhir Dhage, Head of the Department of Computer Engineering, Sardar Patel Institute of Technology, for his support.

References

[1] L. Lalescu *et al.*, "FET – Free Timetabling Software." [Online]. Available: https://lalescu.ro/liviu/fet/

[2] UniTime, "UniTime – Comprehensive University Timetabling System." [Online]. Available: https://www.unitime.org/

[3] Applied Software Consultants, "aSc Timetables." [Online]. Available: https://www.asctimetables.com/

[4] Tabula, "Tabula: Extract tables from PDFs." [Online]. Available: https://tabula.technology/

[5] Camelot, "Camelot: PDF table extraction for humans." [Online]. Available: https://camelot-py.readthedocs.io/

[6] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023, pp. 79–90.

[7] R. Pedro, D. Castro, P. Carreira and N. Santos, "From prompt injections to SQL injection attacks: How protected is your LLM-integrated web application?," *arXiv preprint arXiv:2308.01990*, 2023.

[8] A. Schaerf, "A survey of automated timetabling," *Artificial Intelligence Review*, vol. 13, no. 2, pp. 87–127, 1999.

[9] E. K. Burke and S. Petrovic, "Recent research directions in automated timetabling," *European Journal of Operational Research*, vol. 140, no. 2, pp. 266–280, 2002.

[10] R. Lewis, "A survey of metaheuristic-based techniques for university timetabling problems," *OR Spectrum*, vol. 30, no. 1, pp. 167–190, 2008.

[11] R. Zanibbi, D. Blostein and J. R. Cordy, "A survey of table recognition: Models, observations, transformations, and inferences," *International Journal on Document Analysis and Recognition*, vol. 7, no. 1, pp. 1–16, 2004.

[12] T. Kieninger and A. Dengel, "The T-Recs table recognition and analysis system," in *Document Analysis Systems: Theory and Practice (DAS'98)*, LNCS vol. 1655, Springer, 1999, pp. 255–270.

[13] M. Göbel, T. Hassan, E. Oro and G. Orsi, "ICDAR 2013 table competition," in *Proc. 12th International Conference on Document Analysis and Recognition (ICDAR)*, 2013, pp. 1449–1453.

[14] R. Smith, "An overview of the Tesseract OCR engine," in *Proc. 9th International Conference on Document Analysis and Recognition (ICDAR)*, 2007, pp. 629–633.

[15] I. Androutsopoulos, G. D. Ritchie and P. Thanisch, "Natural language interfaces to databases – an introduction," *Natural Language Engineering*, vol. 1, no. 1, pp. 29–81, 1995.

[16] V. Zhong, C. Xiong and R. Socher, "Seq2SQL: Generating structured queries from natural language using reinforcement learning," *arXiv preprint arXiv:1709.00103*, 2017.

[17] T. Yu *et al.*, "Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task," in *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018, pp. 3911–3921.

[18] J. F. Allen, "Maintaining knowledge about temporal intervals," *Communications of the ACM*, vol. 26, no. 11, pp. 832–843, 1983.

[19] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA: MIT Press, 2009.

[20] R. S. Sandhu, E. J. Coyne, H. L. Feinstein and C. E. Youman, "Role-based access control models," *IEEE Computer*, vol. 29, no. 2, pp. 38–47, 1996.

[21] A. Biryukov, D. Dinu and D. Khovratovich, "Argon2: New generation of memory-hard functions for password hashing and other applications," in *Proc. IEEE European Symposium on Security and Privacy (EuroS&P)*, 2016, pp. 292–302.

[22] M. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015.

[23] OWASP Foundation, "OWASP Top 10:2021," 2021. [Online]. Available: https://owasp.org/Top10/

[24] Meta Platforms, "React Native documentation." [Online]. Available: https://reactnative.dev/

[25] Expo, "Expo documentation (SDK 58)." [Online]. Available: https://docs.expo.dev/

[26] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, University of California, Irvine, 2000.

[27] S. Ramírez, "FastAPI documentation." [Online]. Available: https://fastapi.tiangolo.com/

[28] The PostgreSQL Global Development Group, "PostgreSQL 16 Documentation – SELECT: The locking clause (FOR UPDATE, SKIP LOCKED)." [Online]. Available: https://www.postgresql.org/docs/16/sql-select.html

[29] E. F. Codd, "A relational model of data for large shared data banks," *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, 1970.

[30] A. Silberschatz, H. F. Korth and S. Sudarshan, *Database System Concepts*, 7th ed. New York: McGraw-Hill, 2019.

[31] Render, "Render documentation – Free instance types and free PostgreSQL." [Online]. Available: https://render.com/docs/free

[32] Artifex Software, "PyMuPDF documentation." [Online]. Available: https://pymupdf.readthedocs.io/

7.2 Poster

The A3 project poster is reproduced on the next page.

Sardar Patel Institute of Technology
Department of Computer Engineering
Academic Year 2026-27

Dynamic Timetable: A Role-Based Timetable Monitoring System with Automated Extraction and Natural-Language Queries

Shivam Mulani, Gurshaan Nandrajog, Harshvarddhan Waghela, B.Tech.(Comp.)
Project Guide: Prof. Anant Nimkar | Group No. 86

Abstract

College timetables are published as dense multi-page PDFs that everyone reads by hand. We built Dynamic Timetable, a role-based mobile and web system that extracts the o cial timetable automatically (PDF, scans, Excel, Word), ags uncertain entries, detects clashes and answers plain-English questions through a safe rule-based engine. On the college's real 10-page timetable, 452 of 453 classes were veri ed automatically, exactly 2 genuine clashes were found, and a search takes about 40 ms.

Introduction

The department publishes its class timetable as a 10-page PDF: grids per division with stacked lab batches (A1-A4), teacher codes and page-wise legends. Everyone reads it by hand many times a day, and clashes are noticed only when they happen.

Objective(s)

- Read the real timetable automatically; ag every uncertain value
- Role-based views for Student, Professor, HOD, Principal, Admin
- Answer plain-English questions safely; detect clashes
- One app for Android, iPhone and laptop; free online demo

Problem De nition

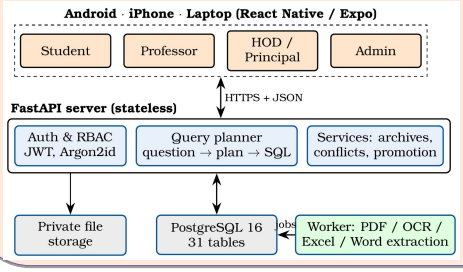
Turn a human-formatted timetable (merged and stacked cells, missing a.m./p.m., legends, scans) into veri ed data, and give each user correct, role-scoped answers without guessing and without exposing personal data.

Contribution

- Timetable-aware parser (backtracking time-row resolver, lab groups, legends)
- Clash detector: 542 raw overlaps 2 genuine clashes
- Compositional query planner: question typed plan safe SQL (no xed question list)

Design

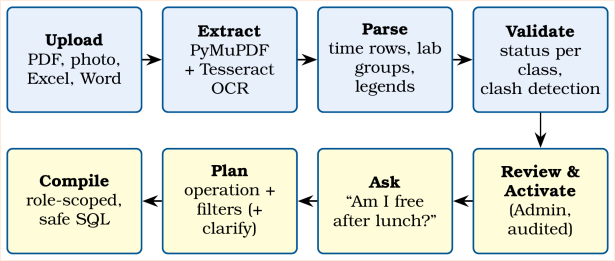
React Native app stateless FastAPI server PostgreSQL, plus a background extraction worker.



Scan: source code

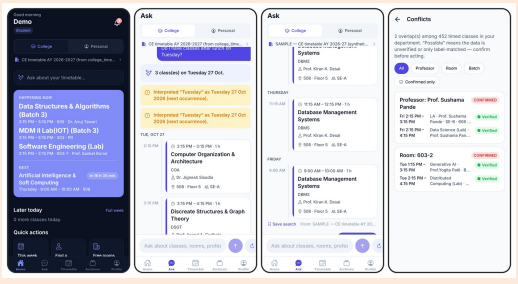
Methodology/Algorithms

1. Upload: text with coordinates (PyMuPDF), or OCR (Tesseract) for scans and photos.
2. Time rows resolved by constrained backtracking; only a unique valid reading is accepted.
3. Cells split into lab groups, legend codes expanded; every class gets a status.
4. Clashes: group by teacher/room/batch, sort, sweep; overlap if s1 < e2 and s2 < e1.
5. Question query plan (count/ rst/last/per-day + lters) clarify if ambiguous safe SQL.



Results

- Real PDF: 453 classes, 452 veri ed (99.8%); the remaining one has no time printed in the PDF
- Exactly 2 genuine clashes reported (a teacher on Friday, room 603-2 on Tuesday)
- Search 37-47 ms (median); 0 wrong answers with 32 concurrent users
- 10-page PDF extracted in 1.65 s; 179 + 20 automated tests and end-to-end tests pass



Conclusion

Dynamic Timetable turns the published PDF into a trustworthy, searchable app for every role. Showing uncertainty instead of guessing kept every automatic answer correct, and the clash report found real errors in the o cial timetable.

References

[1] R. Zanibbi et al., "A survey of table recognition," IJDAR, 2004.
[2] R. Smith, "An overview of the Tesseract OCR engine," ICDAR, 2007.
[3] I. Androutsopoulos et al., "Natural language interfaces to databases," Nat. Lang. Eng., 1995.

Chapter 8: Plagiarism Report

8.1 Plagiarism Report

Plagiarism report to be inserted here
(`figures/plagiarism-report.pdf`)

8.2 AI Content Detection Report

AI content detection report to be inserted here
(`figures/ai-detection-report.pdf`)

References

[1] A. Schaerf, "A survey of automated timetabling," *Artificial Intelligence Review*, vol. 13, no. 2, pp. 87–127, 1999.

[2] E. K. Burke and S. Petrovic, "Recent research directions in automated timetabling," *European Journal of Operational Research*, vol. 140, no. 2, pp. 266–280, 2002.

[3] R. Lewis, "A survey of metaheuristic-based techniques for university timetabling problems," *OR Spectrum*, vol. 30, no. 1, pp. 167–190, 2008.

[4] R. Zanibbi, D. Blostein and J. R. Cordy, "A survey of table recognition: Models, observations, transformations, and inferences," *International Journal on Document Analysis and Recognition*, vol. 7, no. 1, pp. 1–16, 2004.

[5] T. Kieninger and A. Dengel, "The T-Recs table recognition and analysis system," in *Document Analysis Systems: Theory and Practice (DAS'98)*, LNCS vol. 1655, Springer, 1999, pp. 255–270.

[6] M. Göbel, T. Hassan, E. Oro and G. Orsi, "ICDAR 2013 table competition," in *Proc. 12th International Conference on Document Analysis and Recognition (ICDAR)*, 2013, pp. 1449–1453.

[7] R. Smith, "An overview of the Tesseract OCR engine," in *Proc. 9th International Conference on Document Analysis and Recognition (ICDAR)*, 2007, pp. 629–633.

[8] I. Androutsopoulos, G. D. Ritchie and P. Thanisch, "Natural language interfaces to databases – an introduction," *Natural Language Engineering*, vol. 1, no. 1, pp. 29–81, 1995.

[9] V. Zhong, C. Xiong and R. Socher, "Seq2SQL: Generating structured queries from natural language using reinforcement learning," *arXiv preprint arXiv:1709.00103*, 2017.

[10] T. Yu *et al.*, "Spider: A large-scale human-labeled dataset for complex and cross-domain semantic parsing and text-to-SQL task," in *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2018, pp. 3911–3921.

[11] K. Greshake, S. Abdelnabi, S. Mishra, C. Endres, T. Holz and M. Fritz, "Not what you've signed up for: Compromising real-world LLM-integrated applications with indirect prompt injection," in *Proc. 16th ACM Workshop on Artificial Intelligence and Security (AISec)*, 2023, pp. 79–90.

[12] R. Pedro, D. Castro, P. Carreira and N. Santos, "From prompt injections to SQL injection attacks: How protected is your LLM-integrated web application?," *arXiv preprint arXiv:2308.01990*, 2023.

[13] J. F. Allen, "Maintaining knowledge about temporal intervals," *Communications of the ACM*, vol. 26, no. 11, pp. 832–843, 1983.

[14] R. S. Sandhu, E. J. Coyne, H. L. Feinstein and C. E. Youman, "Role-based access control models," *IEEE Computer*, vol. 29, no. 2, pp. 38–47, 1996.

[15] A. Biryukov, D. Dinu and D. Khovratovich, "Argon2: New generation of memory-hard functions for password hashing and other applications," in *Proc. IEEE European Symposium on Security and Privacy (EuroS&P)*, 2016, pp. 292–302.

[16] R. T. Fielding, "Architectural styles and the design of network-based software architectures," Ph.D. dissertation, University of California, Irvine, 2000.

[17] E. F. Codd, "A relational model of data for large shared data banks," *Communications of the ACM*, vol. 13, no. 6, pp. 377–387, 1970.

[18] M. Jones, J. Bradley and N. Sakimura, "JSON Web Token (JWT)," IETF RFC 7519, May 2015.

[19] OWASP Foundation, "OWASP Top 10:2021," 2021. [Online]. Available: https://owasp.org/Top10/

[20] A. Silberschatz, H. F. Korth and S. Sudarshan, *Database System Concepts*, 7th ed. New York: McGraw-Hill, 2019.

[21] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, *Introduction to Algorithms*, 3rd ed. Cambridge, MA: MIT Press, 2009.

[22] A. Silberschatz, P. B. Galvin and G. Gagne, *Operating System Concepts*, 10th ed. Hoboken, NJ: Wiley, 2018.

[23] I. Sommerville, *Software Engineering*, 10th ed. Boston: Pearson, 2016.

[24] L. Lalescu *et al.*, "FET – Free Timetabling Software." [Online]. Available: https://lalescu.ro/liviu/fet/

[25] UniTime, "UniTime – Comprehensive University Timetabling System." [Online]. Available: https://www.unitime.org/

[26] Applied Software Consultants, "aSc Timetables." [Online]. Available: https://www.asctimetables.com/

[27] Tabula, "Tabula: Extract tables from PDFs." [Online]. Available: https://tabula.technology/

[28] Camelot, "Camelot: PDF table extraction for humans." [Online]. Available: https://camelot-py.readthedocs.io/

[29] Artifex Software, "PyMuPDF documentation." [Online]. Available: https://pymupdf.readthedocs.io/

[30] The PostgreSQL Global Development Group, "PostgreSQL 16 Documentation – SELECT: The locking clause (FOR UPDATE, SKIP LOCKED)." [Online]. Available: https://www.postgresql.org/docs/16/sql-select.html

[31] S. Ramírez, "FastAPI documentation." [Online]. Available: https://fastapi.tiangolo.com/

[32] Expo, "Expo documentation (SDK 58)." [Online]. Available: https://docs.expo.dev/

[33] Meta Platforms, "React Native documentation." [Online]. Available: https://reactnative.dev/

[34] Render, "Render documentation – Free instance types and free PostgreSQL." [Online]. Available: https://render.com/docs/free